LAPORAN TUGAS BESAR 1

IF3070 DASAR INTELIGENSI ARTIFISIAL

OPTIMASI PENEMPATAN KENDARAAN PADA GELADAK KAPAL KARGO MENGGUNAKAN ALGORITMA LOCAL SEARCH: HILL CLIMBING, SIMULATED ANNEALING, DAN GENETIC ALGORITHM



Dosen Pengampu:

Ir. Windy Gambetta, MBA

Dr. Nur Ulfa Maulidevi, S.T, M.Sc.

Disusun oleh:

Kelompok 04 - Brimanuy

Danya Damaris	18224024
Lukman Hakim Syah Ardhana	18224097
Jacko Luciano	18224101

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **1**
DAFTAR TABEL **2**
DAFTAR GAMBAR **3**
BAB I: DESKRIPSI PERSOALAN **9**
1.1 Permasalahan 9
1.2 Spesifikasi 9
BAB II: PEMBAHASAN **11**
2.1 Implementasi 11
2.1.1 Hill-Climbing 15
2.1.2 Simulated Annealing 17
2.1.3 Genetic Algorithm 19
2.2 Analisis Hasil 20
2.2.1 Hill-Climbing 21
2.2.2 Simulated Annealing 31
2.2.3 Genetic Algorithm 35
2.2.4 Perbandingan Antar Algoritma 50
2.2.5 Pengujian Bonus 51
BAB III: KESIMPULAN DAN SARAN **69**
3.1 Kesimpulan 69
3.2 Saran 70
PEMBAGIAN TUGAS **71**
REFERENSI **72**
LAMPIRAN **73**

DAFTAR TABEL

Tabel 1.2.1 Entitas dan Atribut Permasalahan Penempatan Kendaraan..... 10
Tabel 2.2.1.1 Hasil Pengujian Empat Varian Hill-Climbing pada Case1.....21
Tabel 2.2.1.2 Jumlah Iterasi Setiap Run pada Random Restart Hill-Climbing.....21
Tabel 2.2.1.3 Visualisasi State Awal dan Akhir Empat Varian Hill-Climbing.....22
Tabel 2.2.1.4 Kurva Nilai Objective Empat Varian Hill-Climbing.....25
Tabel 2.2.2.1 Hasil Pengujian Simulated Annealing pada Case1..... 31
Tabel 2.2.2.2 Visualisasi State Awal dan Akhir Simulated Annealing..... 32
Tabel 2.2.2.3 Kurva Nilai Objective dan Rasio Penerimaan Simulated Annealing.....33
Tabel 2.2.3.1 Hasil Pengujian Genetic Algorithm dengan Variasi Jumlah Generasi pada Populasi 50.35
Tabel 2.2.3.2 Hasil Pengujian Genetic Algorithm dengan Variasi Ukuran Populasi pada 200 Generasi.. 36
Tabel 2.2.3.3 Ringkasan Hasil Pengujian Seluruh Konfigurasi Genetic Algorithm.....36
Tabel 2.2.3.4 Visualisasi State Awal dan Akhir Genetic Algorithm dengan Variasi Jumlah Generasi..37
Tabel 2.2.3.5 Kurva Fitness Maksimum dan Rata-Rata Genetic Algorithm dengan Variasi Jumlah Generasi.....40
Tabel 2.2.3.6 Visualisasi State Awal dan Akhir Genetic Algorithm dengan Variasi Ukuran Populasi..43
Tabel 2.2.3.7 Kurva Fitness Maksimum dan Rata-Rata Genetic Algorithm dengan Variasi Ukuran Populasi.....46
Tabel 2.2.3.8 Perbandingan Kinerja Algoritma pada Case1 dengan Mode Fee..... 50
Tabel 2.2.3.9 Hasil Pengujian Algoritma pada Kasus Multikapal..... 51
Tabel 2.2.3.10 Visualisasi State Awal dan Akhir pada Kasus Multikapal..... 52
Tabel 2.2.3.11 Hasil Pengujian Algoritma dengan Objective Function Mode Urgency.....56
Tabel 2.2.3.12 Visualisasi State Awal dan Akhir pada Mode Urgency.....57
Tabel 2.2.3.13 Hasil Pengujian Algoritma dengan Objective Function Mode Area.....62
Tabel 2.2.3.14 Visualisasi State Awal dan Akhir pada Mode Area.....62

DAFTAR GAMBAR

Gambar 1.2.1 5
Gambar 2.1.1 10
Gambar 2.1.2 11
Gambar 2.2.1.1 18
Gambar 2.2.1.2 18
Gambar 2.2.1.3 18
Gambar 2.2.1.4 18
Gambar 2.2.1.5 18
Gambar 2.2.1.6 18
Gambar 2.2.1.7 19
Gambar 2.2.1.8 19
Gambar 2.2.1.9 19
Gambar 2.2.1.10 19
Gambar 2.2.1.11 19
Gambar 2.2.1.12 19
Gambar 2.2.1.13 20
Gambar 2.2.1.14 20
Gambar 2.2.1.15 20
Gambar 2.2.1.16 20
Gambar 2.2.1.17 20
Gambar 2.2.1.18 20
Gambar 2.2.1.19 21
Gambar 2.2.1.20 21
Gambar 2.2.1.21 21
Gambar 2.2.1.22 21
Gambar 2.2.1.23 21
Gambar 2.2.1.24 21
Gambar 2.2.1.25 22
Gambar 2.2.1.26 22
Gambar 2.2.1.27 22
Gambar 2.2.1.28 23
Gambar 2.2.1.29 23
Gambar 2.2.1.30 23
Gambar 2.2.1.31 24
Gambar 2.2.1.32 24
Gambar 2.2.1.33 24
Gambar 2.2.1.34 25
Gambar 2.2.1.35 25
Gambar 2.2.1.36 25
Gambar 2.2.2.1 28
Gambar 2.2.2.2 28
Gambar 2.2.2.3 28
Gambar 2.2.2.4 28

Gambar 2.2.2.5....28
Gambar 2.2.2.6....28
Gambar 2.2.2.7....29
Gambar 2.2.2.8....29
Gambar 2.2.2.9....29
Gambar 2.2.2.10....29
Gambar 2.2.2.11....29
Gambar 2.2.2.12....29
Gambar 2.2.3.1....33
Gambar 2.2.3.2....33
Gambar 2.2.3.3....33
Gambar 2.2.3.4....33
Gambar 2.2.3.5....33
Gambar 2.2.3.6....33
Gambar 2.2.3.7....34
Gambar 2.2.3.8....34
Gambar 2.2.3.9....34
Gambar 2.2.3.10....34
Gambar 2.2.3.11....34
Gambar 2.2.3.12....34
Gambar 2.2.3.13....35
Gambar 2.2.3.14....35
Gambar 2.2.3.15....35
Gambar 2.2.3.16....35
Gambar 2.2.3.17....35
Gambar 2.2.3.18....35
Gambar 2.2.3.19....36
Gambar 2.2.3.20....36
Gambar 2.2.3.21....36
Gambar 2.2.3.22....37
Gambar 2.2.3.23....37
Gambar 2.2.3.24....37
Gambar 2.2.3.25....38
Gambar 2.2.3.26....38
Gambar 2.2.3.27....38
Gambar 2.2.3.28....39
Gambar 2.2.3.29....39
Gambar 2.2.3.30....39
Gambar 2.2.3.31....39
Gambar 2.2.3.32....39
Gambar 2.2.3.33....39
Gambar 2.2.3.34....40
Gambar 2.2.3.35....40
Gambar 2.2.3.36....40

Gambar 2.2.3.37....40
Gambar 2.2.3.38....40
Gambar 2.2.3.39....40
Gambar 2.2.3.40....41
Gambar 2.2.3.41....41
Gambar 2.2.3.42....41
Gambar 2.2.3.43....41
Gambar 2.2.3.44....41
Gambar 2.2.3.45....41
Gambar 2.2.3.46....42
Gambar 2.2.3.47....42
Gambar 2.2.3.48....42
Gambar 2.2.3.49....43
Gambar 2.2.3.50....43
Gambar 2.2.3.51....43
Gambar 2.2.3.52....44
Gambar 2.2.3.53....44
Gambar 2.2.3.54....44
Gambar 2.2.5.1....48
Gambar 2.2.5.2....48
Gambar 2.2.5.3....48
Gambar 2.2.5.4....48
Gambar 2.2.5.5....48
Gambar 2.2.5.6....48
Gambar 2.2.5.7....48
Gambar 2.2.5.8....48
Gambar 2.2.5.9....48
Gambar 2.2.5.10....48
Gambar 2.2.5.11....49
Gambar 2.2.5.12....49
Gambar 2.2.5.13....49
Gambar 2.2.5.14....49
Gambar 2.2.5.15....49
Gambar 2.2.5.16....49
Gambar 2.2.5.17....49
Gambar 2.2.5.18....49
Gambar 2.2.5.19....49
Gambar 2.2.5.20....49
Gambar 2.2.5.21....49
Gambar 2.2.5.22....49
Gambar 2.2.5.23....50
Gambar 2.2.5.24....50
Gambar 2.2.5.25....50
Gambar 2.2.5.26....50

Gambar 2.2.5.27 50
Gambar 2.2.5.28 50
Gambar 2.2.5.29 50
Gambar 2.2.5.30 50
Gambar 2.2.5.31 50
Gambar 2.2.5.32 50
Gambar 2.2.5.33 50
Gambar 2.2.5.34 50
Gambar 2.2.5.35 51
Gambar 2.2.5.36 51
Gambar 2.2.5.37 51
Gambar 2.2.5.38 51
Gambar 2.2.5.39 51
Gambar 2.2.5.40 51
Gambar 2.2.5.41 51
Gambar 2.2.5.42 51
Gambar 2.2.5.43 52
Gambar 2.2.5.44 52
Gambar 2.2.5.45 52
Gambar 2.2.5.46 52
Gambar 2.2.5.47 52
Gambar 2.2.5.48 52
Gambar 2.2.5.49 53
Gambar 2.2.5.50 53
Gambar 2.2.5.51 53
Gambar 2.2.5.52 53
Gambar 2.2.5.53 53
Gambar 2.2.5.54 53
Gambar 2.2.5.55 53
Gambar 2.2.5.56 53
Gambar 2.2.5.57 54
Gambar 2.2.5.58 54
Gambar 2.2.5.59 54
Gambar 2.2.5.60 54
Gambar 2.2.5.61 54
Gambar 2.2.5.62 54
Gambar 2.2.5.63 54
Gambar 2.2.5.64 54
Gambar 2.2.5.65 55
Gambar 2.2.5.66 55
Gambar 2.2.5.67 55
Gambar 2.2.5.68 55
Gambar 2.2.5.69 55
Gambar 2.2.5.70 55

Gambar 2.2.5.71 55
Gambar 2.2.5.72 55
Gambar 2.2.5.73 56
Gambar 2.2.5.74 56
Gambar 2.2.5.75 56
Gambar 2.2.5.76 56
Gambar 2.2.5.77 56
Gambar 2.2.5.78 56
Gambar 2.2.5.79 56
Gambar 2.2.5.80 56
Gambar 2.2.5.81 57
Gambar 2.2.5.82 57
Gambar 2.2.5.83 57
Gambar 2.2.5.84 57
Gambar 2.2.5.85 58
Gambar 2.2.5.86 58
Gambar 2.2.5.87 58
Gambar 2.2.5.88 58
Gambar 2.2.5.89 58
Gambar 2.2.5.90 58
Gambar 2.2.5.91 58
Gambar 2.2.5.92 58
Gambar 2.2.5.93 59
Gambar 2.2.5.94 59
Gambar 2.2.5.95 59
Gambar 2.2.5.96 59
Gambar 2.2.5.97 59
Gambar 2.2.5.98 59
Gambar 2.2.5.99 59
Gambar 2.2.5.100 59
Gambar 2.2.5.101 60
Gambar 2.2.5.102 60
Gambar 2.2.5.103 60
Gambar 2.2.5.104 60
Gambar 2.2.5.105 60
Gambar 2.2.5.106 60
Gambar 2.2.5.107 60
Gambar 2.2.5.108 60
Gambar 2.2.5.109 61
Gambar 2.2.5.110 61
Gambar 2.2.5.111 61
Gambar 2.2.5.112 61
Gambar 2.2.5.113 61
Gambar 2.2.5.114 61

Gambar 2.2.5.115 61
Gambar 2.2.5.116 61
Gambar 2.2.5.117 62
Gambar 2.2.5.118 62
Gambar 2.2.5.119 62
Gambar 2.2.5.120 62
Gambar 2.2.5.121 62
Gambar 2.2.5.122 62
Gambar 2.2.5.123 62
Gambar 2.2.5.124 62
Gambar 2.2.5.125 63
Gambar 2.2.5.126 63

BAB I: DESKRIPSI PERSOALAN

1.1 Permasalahan

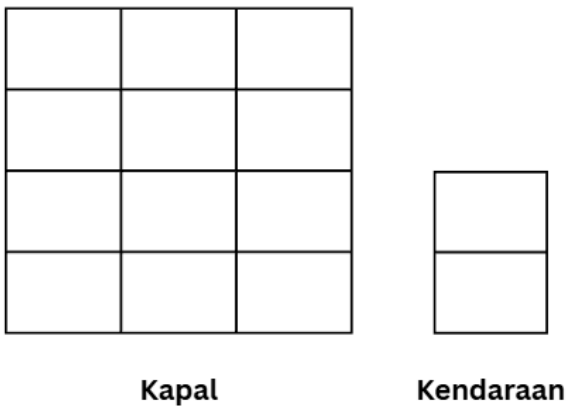
PT Owen Intim Line mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda.

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik dan keputusan pemilihan letak kendaraan belum tentu menghasilkan keuntungan yang optimal.

Tim operasional memerlukan sistem pendukung keputusan untuk menentukan kendaraan yang akan diangkut serta posisi dan orientasi setiap kendaraan pada geladak. Sistem tersebut harus dapat mengevaluasi berbagai kemungkinan konfigurasi, mempertimbangkan fungsi-fungsi objektif yang ditentukan, dan menghasilkan solusi penataan yang optimal.

1.2 Spesifikasi

Sistem bertujuan untuk membuat konfigurasi penempatan kendaraan paling optimal pada sebuah geladak kapal kargo. Setiap kendaraan dan geladak direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang. Solusi dapat memilih kendaraan mana yang diangkut dan menentukan posisi serta orientasinya di dalam geladak, dengan tetap memenuhi seluruh batasan.



Gambar 1.2.1

Daftar variabel permasalahan sebagai berikut:

Tabel 1.2.1 Entitas dan Atribut Permasalahan Penempatan Kendaraan

Entitas	Atribut	Contoh	Keterangan
Vehicle	id	"Owen"	*Identifier*
	Dimensions	{w:3, l:4}	Ukuran sebuah kendaraan, terdiri atas *width* dan *length* (satuan grid)
	Orientation	Horizontal	*Horizontal* atau *Vertical*
	ShippingFee	60	Tarif/keuntungan pengiriman kendaraan (satuan juta rupiah)
	Weight	5	Berat kendaraan (satuan unit)
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan kendaraan (dalam sisa hari)
Ship	Dimensions	{w:10, l:20}	Ukuran geladak pada kapal, terdiri atas *width* dan *length* (satuan grid)
	MaxCapacity	500	Kapasitas angkut maksimum suatu kapal (satuan unit)

Selain daftar variabel, ada beberapa batasan yang ditetapkan agar masalah tidak terlalu kompleks:

1. **Objective function**

   Permasalahan ini bertujuan untuk memaksimalkan *ShippingFee* pada kapal.

2. **Kendaraan tidak boleh saling bertumpuk (*no overlapping*)**

   Tidak boleh ada satu sel grid geladak yang ditempati oleh lebih dari satu kendaraan.

3. **Kendaraan harus berada di dalam batas geladak.**

   Seluruh sel grid yang ditempati suatu kendaraan harus berada di dalam dimensi geladak kapal.

4. **Kapal tidak boleh mengangkut melebihi kapasitasnya.**

BAB II: PEMBAHASAN

2.1 Implementasi

Program mensimulasikan penempatan kendaraan pada geladak kapal kargo dengan tiga keluarga algoritma local search, yaitu Hill-Climbing (seluruh varian), Simulated Annealing, dan Genetic Algorithm. Seluruh algoritma memakai komponen dasar yang sama (representasi state, objective function, dan neighbor), sehingga bagian ini dibahas lebih dulu sebelum masing-masing algoritma.

1. **Representasi state**

   Satu state adalah list berukuran N (jumlah kendaraan). Elemen ke-*i* berupa tuple `(ship_index, x, y, orientasi)`. Nilai `ship_index = -1` (konstanta `OUT`) berarti kendaraan berada di luar kapal. Orientasi `0` adalah horizontal (footprint `w x l`) dan `1` vertical (footprint `l x w`). Koordinat `(x, y)` adalah sel kiri atas kendaraan pada grid geladak. Struktur ini mendukung lebih dari satu kapal (bonus multi-kapal) dan dapat diubah menjadi pemetaan key-value ala JSON melalui `state_to_dict` untuk keperluan visualisasi dan penyimpanan hasil.

```
def footprint(self, i, o):
    v = self.vehicles[i]
    return (v.w, v.l) if o == 0 else (v.l, v.w)
```

2. **Initial state random**

   Setiap kendaraan dengan peluang 30% ditaruh di luar kapal, dan selebihnya ditaruh pada kapal acak, koordinat acak yang masih berada dalam batas geladak, dan orientasi acak. State awal ini belum tentu valid karena tumpang tindih dan kelebihan berat masih mungkin terjadi.

```
def random_placement(problem, i, rng, p_out=0.3, orientation=None):
    o = rng.randint(0, 1) if orientation is None else orientation
    if rng.random() < p_out:
        return (OUT, 0, 0, o)
    s = rng.randrange(len(problem.ships)); ship = problem.ships[s]
    fw, fl = problem.footprint(i, o)
    if fw > ship.w or fl > ship.l:
        return (OUT, 0, 0, o)
    return (s, rng.randint(0, ship.w - fw), rng.randint(0, ship.l - fl), o)
```

3. **Objective function**

   Tujuan utama adalah memaksimalkan total *ShippingFee* kendaraan yang dimuat. Constraint (tidak overlap, tidak keluar batas geladak, tidak melebihi kapasitas) ditangani dengan metode penalti, sehingga state tidak valid tetap dapat dievaluasi dan dilalui oleh algoritma. Nilai objective adalah `score - penalty`. Penalti per satuan pelanggaran ditetapkan `P = 1 + (skor kendaraan tertinggi)`, jadi satu satuan pelanggaran selalu lebih mahal daripada

keuntungan kendaraan mana pun. Pada case1, `P = 101`. Overlap dihitung per sel yang ditempati lebih dari satu kendaraan, kelebihan berat dihitung per unit berat di atas `MaxCapacity`, dan keluar batas dihitung per sel footprint.

```
def evaluate_detail(problem, state):
    P = _pen(problem, problem.mode)
    score, penalty = 0.0, 0.0
    occ = {}
    load = [0] * len(problem.ships)
    for i, (s, x, y, o) in enumerate(state):
        if s == OUT:
            continue
        v = problem.vehicles[i]
        ship = problem.ships[s]
        fw, fl = problem.footprint(i, o)
        if x < 0 or y < 0 or x + fw > ship.w or y + fl > ship.l:
            penalty += P * fw * fl  # keluar batas geladak
        score += vehicle_score(v, problem.mode)
        load[s] += v.weight
        for dx in range(fw):
            for dy in range(fl):
                c = (s, x+dx, y+dy)
                occ[c] = occ.get(c, 0) + 1
    penalty += P * sum(n-1 for n in occ.values() if n > 1) # overlap
    for s, ship in enumerate(problem.ships):
        penalty += P * max(0, load[s] - ship.max_capacity) # overweight
    return score, penalty
```

Untuk bonus *objective function* alternatif, `vehicle_score` menyediakan tiga mode tanpa menambah variabel baru: `fee` (tarif saja), `urgency` (`fee * (1 + 1/(1 + ETA))`, makin dekat ETA makin besar bobotnya), dan `area` (`fee + 0.5 * w * l`). Seluruh eksperimen pada bab ini memakai mode `fee`, mode lain diuji pada subbab 2.2.5.

4. Neighbor

Sesuai spesifikasi terdapat tiga jenis langkah: menukar dua kendaraan, memindahkan kendaraan ke koordinat berbeda (termasuk ke luar kapal), dan memutar orientasi. Satu move hanya memiliki 1 jenis langkah. Pada penukaran, dua kendaraan saling bertukar posisi `(kapal, x, y)` dengan orientasi masing-masing tetap. Jika salah satu hasil tukar keluar dari batas geladak atau tidak muat, penukaran dibatalkan. Rotasi diperlakukan sama: jika footprint hasil rotasi keluar batas, rotasi dibatalkan. Pada langkah pemindahan, kendaraan ditempatkan ulang secara acak (termasuk kemungkinan ke luar kapal) dengan orientasi lamanya dipertahankan. Karena langkah yang dibatalkan atau kebetulan menghasilkan penempatan yang sama akan menghasilkan neighbor yang identik dengan state asal, `random_neighbor` mencoba ulang hingga 20 kali sampai diperoleh state yang berbeda. Jika semua percobaan gagal, neighbor pertama yang berbeda dari `all_neighbors` dipakai (atau salinan state jika memang tidak ada neighbor). Dengan demikian iterasi SA, Stochastic HC, dan mutasi GA

tidak terbuang pada langkah kosong. Terdapat dua generator: `random_neighbor` yang menghasilkan satu neighbor acak (dipakai SA, Stochastic HC, dan mutasi GA), serta `all_neighbors` yang menghasilkan seluruh neighbor (dipakai Steepest Ascent, Sideways Move, dan Random Restart).

```
def random_neighbor(problem, state, rng):
    n = len(state)

    for _ in range(20):
        new = state[:]
        kind = rng.choice(("move", "swap", "rotate"))

        if kind == "swap" and n >= 2:
            i, j = rng.sample(range(n), 2)
            (si, xi, yi, oi), (sj, xj, yj, oj) = state[i], state[j]
            pi, pj = (sj, xj, yj, oi), (si, xi, yi, oj)
            if _within_bounds(problem, i, pi) and _within_bounds(problem, j, pj):
                new[i], new[j] = pi, pj

        elif kind == "rotate":
            i = rng.randrange(n); s, x, y, o = state[i]
            placement = (s, x, y, 1 - o)
            if _within_bounds(problem, i, placement):
                new[i] = placement

        else:
            i = rng.randrange(n)
            new[i] = random_placement(
                problem, i, rng, orientation=state[i][3]
            )
        if new != state:
            return new

    return next(
        (nb for nb in all_neighbors(problem, state) if nb != state),
        state[:]
    )
```

`all_neighbors` melakukan enumerasi penuh. Untuk tiap kendaraan dibangkitkan (1) rotasi di tempat bila footprint hasil rotasi masih muat, dan (2) pemindahan ke `OUT` serta ke seluruh posisi yang muat pada setiap kapal dengan orientasi saat ini. Ditambah (3) seluruh pasangan penukaran yang kedua hasilnya masih berada dalam batas geladak. Pada state awal seed 0 case1 terdapat 2.120 neighbor, dan jumlah ini menentukan biaya waktu varian Hill-Climbing yang dibahas di bagian analisis.

5. Heuristik

Beberapa heuristik dipakai untuk menjaga pencarian tetap efektif dalam lingkup local search.

a. Constraint ditangani dengan metode penalti `P = 1 + skor` kendaraan tertinggi, sehingga state tidak valid tetap bisa dilalui tetapi selalu kalah dari state valid yang setara.

b. Seluruh algoritma tidak mengembalikan state terakhir, melainkan arsip solusi feasible terbaik yang pernah dikunjungi, dengan fallback semua kendaraan di luar kapal jika tidak ada yang feasible. Random Restart mengembalikan hasil terbaik di antara seluruh run steepest-nya.

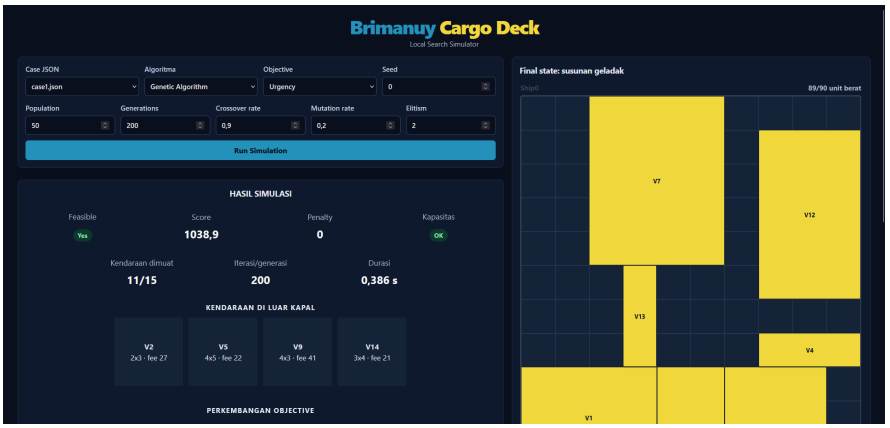
c. `random_neighbor` mencoba ulang hingga 20 kali agar tidak menghasilkan neighbor identik dengan state asal, sehingga iterasi tidak terbuang.

d. Sideways Move memilih neighbor sama nilai secara acak dengan reservoir sampling agar seragam.

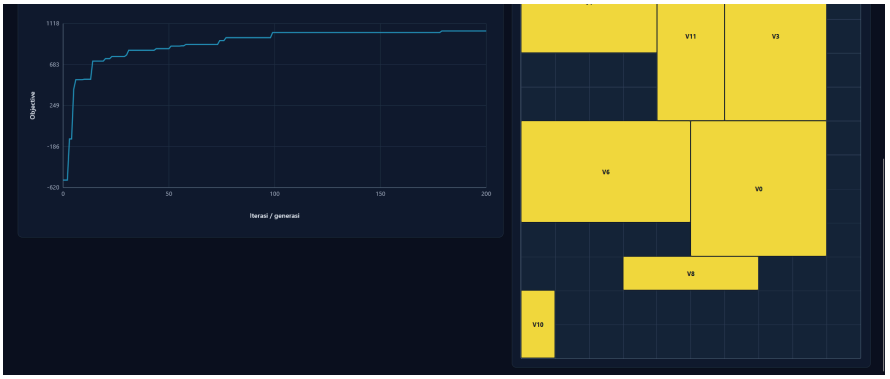
e. Setiap kendaraan punya peluang 30% diletakkan di luar kapal pada state acak, supaya pencarian dimulai dari state yang tidak terlalu padat dan tumpang tindih.

6. Visualisasi

Program dibuat dalam bentuk antarmuka web berbasis FastAPI dan python web yang menampilkan susunan geladak, kendaraan di luar kapal, ringkasan constraint, nilai akhir, jumlah iterasi, durasi, dan grafik perkembangan objective. Visualisasi *initial state*, *final state*, dan kurva objective juga disimpan sebagai gambar oleh skrip eksperimen.



Gambar 2.1.1



Gambar 2.1.2

2.1.1 *Hill-Climbing*

Kami mengimplementasikan keempat varian Hill-Climbing: Steepest Ascent, Sideways Move, Stochastic, dan Random Restart. Semuanya berstatus *maximization*: sebuah langkah hanya diterima jika meningkatkan (atau, pada sideways, menyamai) nilai objective.

1. **Steepest Ascent**

   Pada tiap iterasi seluruh neighbor dievaluasi dan dipilih neighbor terbaik. Pencarian berhenti ketika tidak ada neighbor dengan nilai yang lebih besar dari state saat ini. Banyak iterasi dicatat pada `Result.iterations`.

```
def steepest_ascent(problem, init, rng=None):
    t0 = time.time()
    cur, cur_v = init[:], evaluate(problem, init)
    best_feasible = cur[:] if is_feasible(problem, cur) else None
    best_feasible_v = cur_v if best_feasible is not None else
float("-inf")
    hist, it = [cur_v], 0
    while True:
        best, best_v = None, cur_v
        for nb in all_neighbors(problem, cur):
            v = evaluate(problem, nb)
            if v > best_v:
                best, best_v = nb, v
        if best is None:
            break
        cur, cur_v = best, best_v
        if cur_v > best_feasible_v and is_feasible(problem, cur):
            best_feasible, best_feasible_v = cur[:], cur_v
        it += 1; hist.append(cur_v)
    final = ([(OUT, 0, 0, p[3]) for p in init]
             if best_feasible is None else best_feasible)
    return Result("steepest", init, final, evaluate(problem, final),
                  hist, it, time.time() - t0)
```

   Solusi yang dikembalikan adalah state feasible terbaik yang pernah dikunjungi, sehingga hasil akhir selalu valid.

2. **Hill-Climbing with Sideways Move**

   Varian ini menambahkan parameter `max_sideways` (default 100). Jika tidak ada neighbor yang lebih baik tetapi ada neighbor bernilai sama, salah satunya dipilih secara acak (reservoir sampling agar seragam di antara kandidat sama nilai) dan dihitung sebagai satu sideways move. Penghitung sideways direset ke 0 setiap kali ditemukan perbaikan, sehingga batas berlaku untuk sideways move berturut-turut. Pencarian dihentikan ketika batas tercapai atau tidak ada neighbor sama nilai.

```
       for nb in all_neighbors(problem, cur):
            v = evaluate(problem, nb)
```

```
            if v > best_v:
                best, best_v = nb, v
            elif v == cur_v:
                n_eq += 1
                if rng.random() < 1 / n_eq:
                    eq = nb
        if best is not None:
            cur, cur_v, side = best, best_v, 0
        elif eq is not None and side < max_sideways:
            cur, side = eq, side + 1
        else:
            break
```

3. **Stochastic Hill-Climbing.**

Pada tiap iterasi hanya satu neighbor acak (`random_neighbor`) yang dibangkitkan, dan diterima bila nilainya lebih baik. Karena tidak ada kondisi berhenti alami (neighbor yang ditolak bukan berarti tidak ada neighbor lebih baik), pencarian dibatasi oleh `max_iter = 20000` dan *patience* 2000 iterasi berturut-turut tanpa perbaikan.

```
for it in range(1, max_iter + 1):
     nb= random_neighbor(problem, cur, rng); v=evaluate(problem,nb)
     if v > cur_v:
         cur, cur_v, no_imp = nb, v, 0
     else:
         no_imp += 1
     hist.append(cur_v)
     if no_imp >= patience:
         break
```

4. **Random Restart Hill-Climbing.**

Steepest Ascent dijalankan berulang. Run pertama berangkat dari initial state, dan tiap restart berikutnya dari state acak baru. Parameter `max_restart` (default 10) membatasi jumlah restart, sehingga total ada 11 run steepest. Solusi akhir adalah yang terbaik di antara seluruh run. Jumlah restart dan iterasi per restart disimpan pada `extras`, dan `history` merupakan gabungan riwayat seluruh run.

```
for r in range(max_restart + 1):   # run ke-0 = initial state
     res = steepest_ascent(problem, state)
     iters.append(res.iterations); all_hist += res.history
     if best is None or res.final_value > best.final_value:
         best = res
     state = random_state(problem, rng)
```

2.1.2 *Simulated Annealing*

Simulated Annealing memakai satu neighbor acak per iterasi. Neighbor yang lebih baik (`ΔE > 0`) selalu diterima, sedangkan neighbor yang lebih buruk atau sama diterima dengan peluang `e^(ΔE/T)`. Jadwal pendinginan bersifat geometrik: `T ← α·T` pada tiap iterasi dengan parameter default `T0 = 60`, `α = 0.999`, `T_min = 0.01`, dan `max_iter = 100000`. Dengan parameter tersebut pencarian berhenti ketika `T` mencapai `T_min` setelah sekitar 8.700 iterasi (terukur 8.696). Nilai `T0 = 60` dipilih agar sepadan dengan skala tarif kendaraan (10–100), sehingga pada awal pencarian langkah yang menurunkan nilai sekitar puluhan unit masih cukup sering diterima.

Solusi yang dikembalikan adalah solusi feasible (penalti 0) terbaik yang pernah dikunjungi, bukan sekadar state terakhir dan bukan state terbaik mentah yang bisa saja melanggar constraint. Setiap kali sebuah state diterima, program memeriksa apakah state tersebut feasible dan lebih baik dari arsip feasible sebelumnya. Jika tidak pernah ditemukan state feasible, solusi cadangan berupa semua kendaraan di luar kapal (nilai 0) dikembalikan dan ditandai `fallback_used`. State terbaik mentah tetap disimpan pada `extras["best_raw_state"]` dan `extras["best_raw_value"]` untuk keperluan analisis. Untuk kebutuhan analisis lain, setiap peluang `e^(ΔE/T)` yang dihitung dicatat pada `extras["probs"]` (untuk plot), dan `rejected_worse` menghitung berapa kali langkah yang memburuk ditolak.

Frekuensi stuck di local optimum diukur secara langsung: setiap `check_every = 50` iterasi, fungsi `is_local_optimum` memeriksa apakah tidak ada neighbor dari `all_neighbors` yang bernilai lebih baik daripada state saat ini. Jumlah pengecekan (`lo_checks`), jumlah yang ternyata local optimum (`lo_hits`), dan log-nya (`lo_log`: iterasi, nilai, suhu) disimpan pada extras. Frekuensi stuck adalah `lo_hits` / `lo_checks`. Waktu pengecekan ini dicatat terpisah pada `extras["local_optimum_check_seconds"].` Field `Result.duration` mencakup pengecekan tersebut, sedangkan waktu pencarian murni disimpan pada `extras["search_duration_seconds"]` (durasi total dikurangi waktu pengecekan).

```
def is_local_optimum(problem, state, value):
    # True jika tidak ada neighbor yang nilainya lebih baik dari state saat ini.
     return not any(evaluate(problem, nb) > value for nb in all_neighbors(problem,
state))

def simulated_annealing(problem, init, rng, T0=60.0, alpha=0.999, T_min=0.01,
                        max_iter=100000, check_every=50):
    t0 = time.time()
    cur, cur_v = init[:], evaluate(problem, init)
    best, best_v = cur, cur_v

    best_feasible = cur[:] if is_feasible(problem, cur) else None
```

```
    best_feasible_v = cur_v if best_feasible is not None else -math.inf

    hist, probs, rejected_worse = [cur_v], [], 0
    lo_checks, lo_hits, lo_log, t_check = 0, 0, [], 0.0
    T, it = T0, 0
    while T > T_min and it < max_iter:
        it += 1
        nb = random_neighbor(problem, cur, rng); v = evaluate(problem, nb)
        d = v - cur_v
        if d > 0:
            accept = True
        else:
            p = math.exp(d / T)
            probs.append((it, p))
            accept = rng.random() < p
            if not accept:
                rejected_worse += 1
        if accept:
            cur, cur_v = nb, v
            if cur_v > best_v:
                best, best_v = cur, cur_v
            if cur_v > best_feasible_v and is_feasible(problem, cur):
                best_feasible, best_feasible_v = cur[:], cur_v

        hist.append(cur_v)

        # cek local optimum sungguhan tiap check_every iterasi
        if check_every and it % check_every == 0:
            s = time.time()
            lo_checks += 1
            if is_local_optimum(problem, cur, cur_v):
                lo_hits += 1
                lo_log.append((it, cur_v, T))   # (iterasi, nilai, suhu)
            t_check += time.time() - s
        T *= alpha

    duration = time.time() - t0

    fallback_used = best_feasible is None
    final = ([(OUT, 0, 0, p[3]) for p in init]
             if fallback_used else best_feasible)
    final_v = evaluate(problem, final)

    return Result(
        "sa", init, final, final_v, hist, it, duration,
        {"probs": probs, "rejected_worse": rejected_worse,
         "lo_checks": lo_checks, "lo_hits": lo_hits, "lo_log": lo_log,
         "check_every": check_every, "T0": T0, "alpha": alpha,
         "best_raw_state": best[:], "best_raw_value": best_v,
         "fallback_used": fallback_used,
         "local_optimum_check_seconds": t_check,
         "search_duration_seconds": max(0.0, duration - t_check)}
    )
```

Perlu dicatat bahwa langkah dengan `ΔE = 0` ikut masuk ke cabang `else` sehingga memiliki peluang `e^0 = 1` dan selalu diterima. Perilaku ini setara dengan sideways move dan membuat SA dapat berpindah di dataran (*plateau*) tanpa hambatan.

2.1.3 *Genetic Algorithm*

Pada Genetic Algorithm, satu kromosom adalah satu state (list placement per kendaraan), dan *fitness* adalah nilai objective berpenalti (`score - penalty`). Parameter yang dapat diubah adalah `pop_size` (jumlah populasi) dan `generations` (banyak iterasi), selain `crossover_rate = 0.9`, `mutation_rate = 0.2`, dan `elitism = 2`.

- **Populasi awal.** Individu pertama adalah initial state yang diberikan, dan sisanya state acak dari `random_state`.
- **Seleksi.** *Roulette wheel*. Karena fitness dapat negatif, bobot digeser menjadi `f - min(fits) + 1` agar selalu positif.
- **Crossover.** *One-point crossover* pada indeks kendaraan: anak pertama mengambil placement kendaraan `0..cut-1` dari induk A dan sisanya dari induk B, anak kedua sebaliknya. Crossover dilakukan dengan peluang `crossover_rate`, jika tidak anak merupakan salinan induk.
- **Mutasi.** Dengan peluang `mutation_rate` sebuah anak diganti dengan `random_neighbor` (move, swap, atau rotate).
- **Elitisme.** Dua individu terbaik tiap generasi disalin langsung ke generasi berikutnya.
- **Arsip solusi feasible.** Solusi akhir adalah individu *feasible* (penalti 0) terbaik sepanjang seluruh generasi. Jika tidak pernah ditemukan individu feasible, solusi cadangan berupa semua kendaraan di luar kapal (nilai 0) dikembalikan dan ditandai `fallback_used`. Pada seluruh eksperimen fallback tidak pernah dipakai.
- **Pencatatan.** Maksimum dan rata-rata fitness populasi, serta proporsi individu feasible, dicatat pada tiap generasi (generasi 0 adalah populasi awal).

```
def roulette_select(pop, fits, rng):
    minimum = min(fits)
    weights = [f - minimum + 1.0 for f in fits]
    total = math.fsum(weights)
    point, cumulative = rng.random() * total, 0.0
    for individual, weight in zip(pop, weights):
        cumulative += weight
        if point < cumulative:
            return individual
    return pop[-1]

def crossover(a, b, rng):
    cut = rng.randint(1, len(a) - 1)
    return a[:cut] + b[cut:], b[:cut] + a[cut:]
```

```
        ranked=sorted(range(pop_size),key=lambda index: fits[index],reverse=True)
        children = [pop[index][:] for index in ranked[:elitism]]
```

```
        while len(children) < pop_size:
            a = roulette_select(pop, fits, rng)
            b = roulette_select(pop, fits, rng)
            pair = (crossover(a, b, rng) if rng.random() < crossover_rate
                    else (a[:], b[:]))
            for child in pair:
                if len(children) == pop_size:
                    break
                if child and rng.random() < mutation_rate:
                    child = random_neighbor(problem, child, rng)
                children.append(child)
        pop = children
```

Skenario eksperimen GA (`experiments/exp_ga.py`):

1. Jumlah populasi dikontrol pada 50 dengan banyak iterasi 50, 150, dan 400
2. Banyak iterasi dikontrol pada 200 dengan jumlah populasi 20, 50, dan 100.

Dengan tiap konfigurasi dijalankan 3 kali (seed 0, 1, 2), sehingga total 18 run.

2.2 Analisis Hasil

Setup eksperimen

Seluruh eksperimen memakai `data/case1.json`: 15 kendaraan (total tarif 851, total berat 138, total luas 165 sel), satu kapal berukuran 10 x 20 (200 sel) dengan `MaxCapacity = 90`, dan mode objective fee, kecuali subbab 2.2.5. Total berat kendaraan (138) jauh melebihi kapasitas (90), sehingga tidak semua kendaraan dapat dimuat dan kendala kapasitas adalah kendala yang aktif. Tiap algoritma dijalankan dengan seed 0, 1, 2. Untuk seed yang sama, seluruh algoritma berangkat dari initial state yang identik, sehingga perbandingan antar algoritma adil. Nilai objective initial state sangat negatif karena penuh pelanggaran: -4.047 (seed 0), -2.529 (seed 1), dan -7.638 (seed 2).

Nilai optimum global. Relaksasi yang hanya memperhatikan berat (masalah knapsack 0/1 dengan kapasitas 90) menghasilkan tarif maksimum 762, dan nilai ini merupakan batas atas bagi masalah asli karena constraint geometri hanya dapat menurunkannya. Solusi bernilai 762 yang valid (feasible) ditemukan oleh beberapa algoritma, sehingga 762 adalah optimum global case1. Solusi ini memuat 11 kendaraan dengan total berat 90 dan membiarkan V2, V4, V5, dan V14 di luar kapal. Seluruh persentase gap di bawah dihitung terhadap 762. (Batas atas 766,25 pada CSV berasal dari relaksasi yang lebih longgar, sehingga 762 tetap menjadi acuan.)

Catatan: durasi dalam bab ini bergantung pada spesifikasi mesin dan beban mesin saat run, sehingga hanya cocok untuk perbandingan kasar. Untuk versi kode yang sama, nilai objective dan iterasi bersifat deterministik per seed. Semua run pada CSV ini feasible dan solusi cadangan (fallback) tidak pernah dipakai.

2.2.1 *Hill-Climbing*

Hasil tiga run tiap varian (case1, mode fee).

Tabel 2.2.1.1 Hasil Pengujian Empat Varian *Hill-Climbing* pada Case1

Varian	Seed	Initial	Final	Gap ke 762	Iterasi	Durasi (s)	Dimuat / berat
Steepest Ascent	0	-4.047	679	10,9%	8	0,80	10 / 90
	1	-2.529	762	0%	11	0,95	11 / 90
	2	-7.638	681	10,6%	8	0,60	10 / 89
Sideways Move (max 100)	0	-4.047	749	1,7%	140	12,39	11 / 89
	1	-2.529	762	0%	111	9,28	11 / 90
	2	-7.638	749	1,7%	169	13,32	11 / 89
Stochastic	0	-4.047	663	13,0%	2.399	0,14	10 / 90
	1	-2.529	653	14,3%	2.649	0,10	9 / 88
	2	-7.638	733	3,8%	2.550	0,11	10 / 84
Random Restart (10 restart)	0	-4.047	762	0%	84	6,71	11 / 90
	1	-2.529	762	0%	83	6,54	11 / 90
	2	-7.638	762	0%	85	6,64	11 / 90

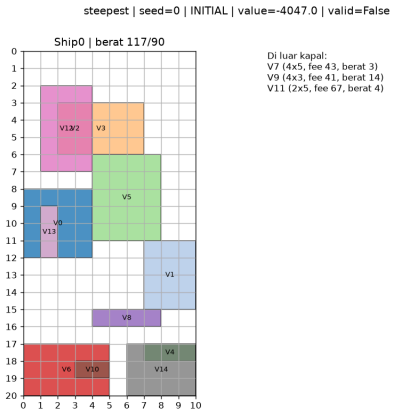
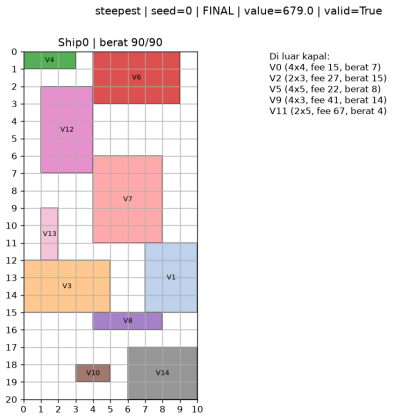
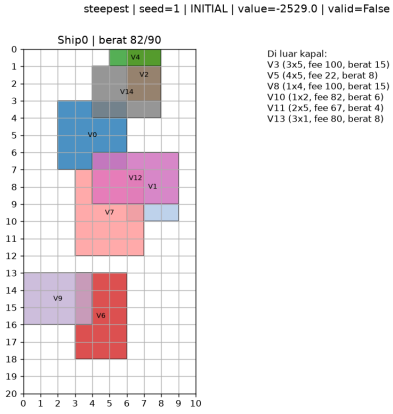
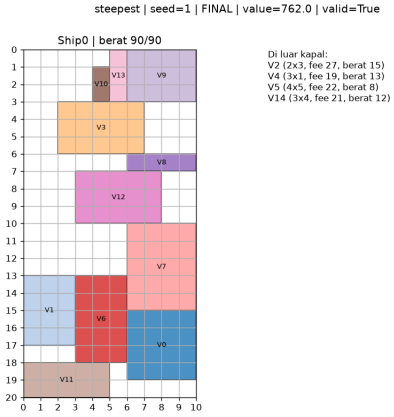
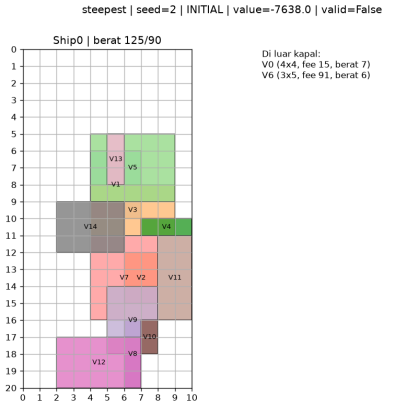
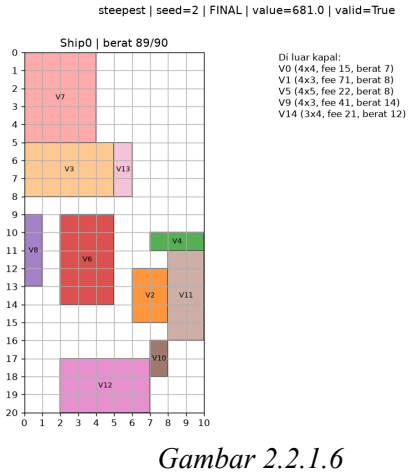
Setiap hasil akhir valid (penalti 0). Banyak iterasi pada Random Restart adalah jumlah iterasi seluruh 11 run steepest (rata-rata sekitar 8 iterasi per run).

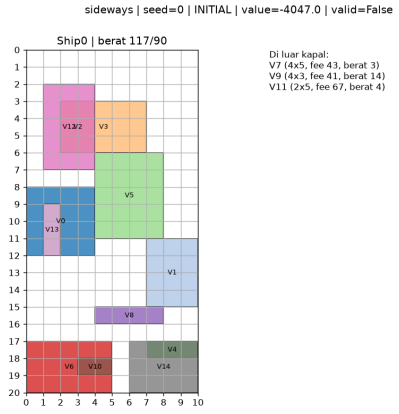
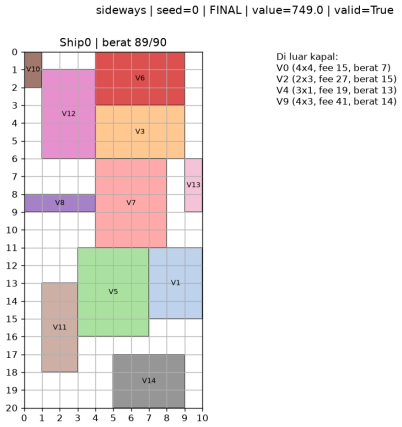
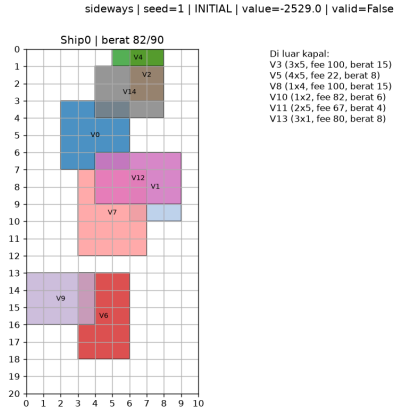
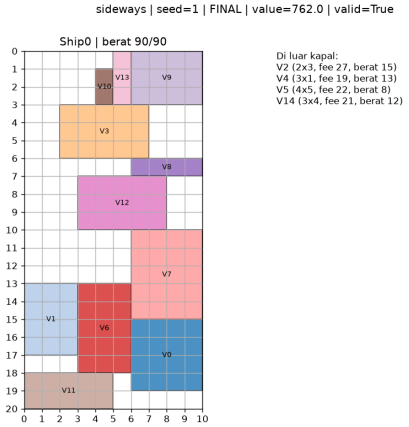
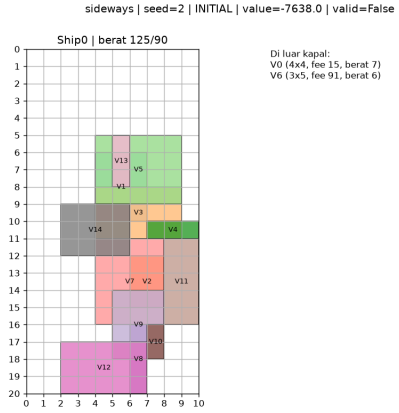
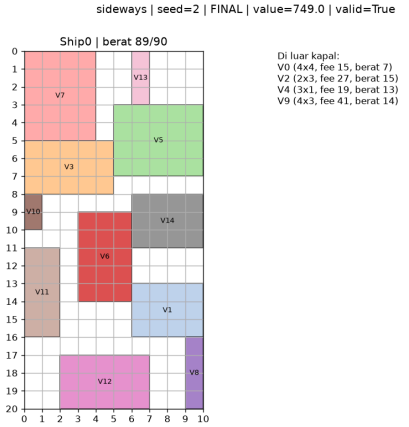
Banyak restart adalah 10 (`max_restart`), sehingga total 11 run steepest. Iterasi tiap run (run ke-0 berangkat dari initial state) ditunjukkan pada tabel berikut.

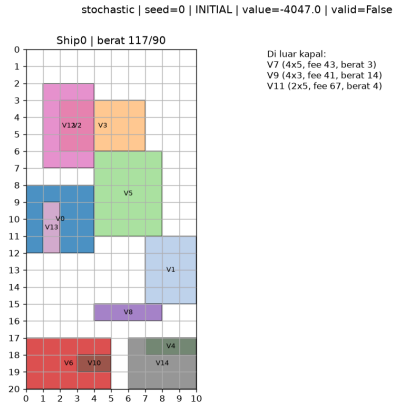
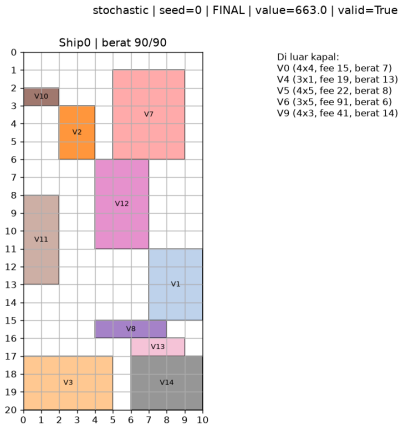
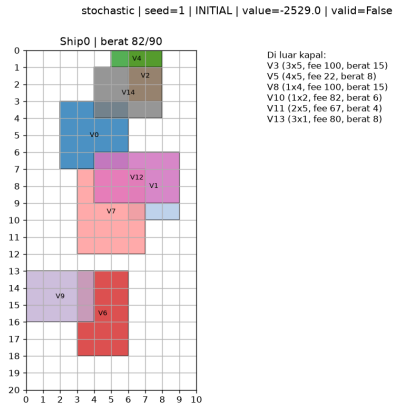
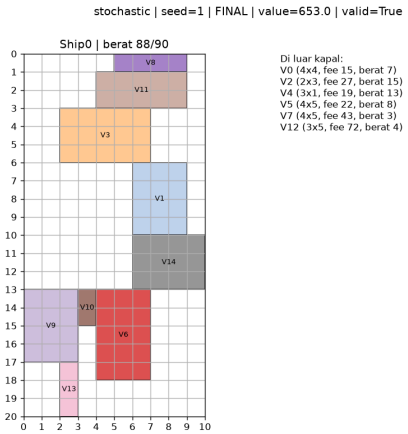
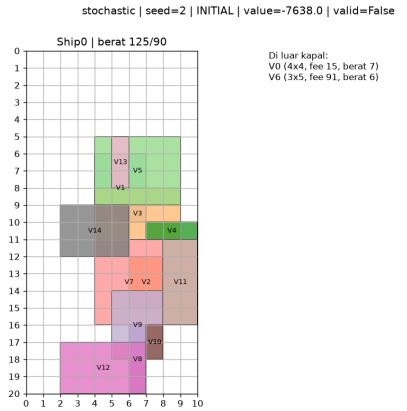
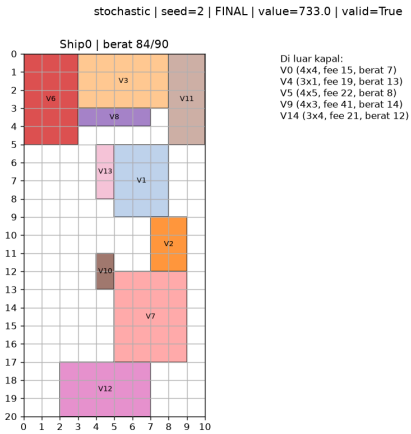
Tabel 2.2.1.2 Jumlah Iterasi Setiap Run pada *Random Restart Hill-Climbing*

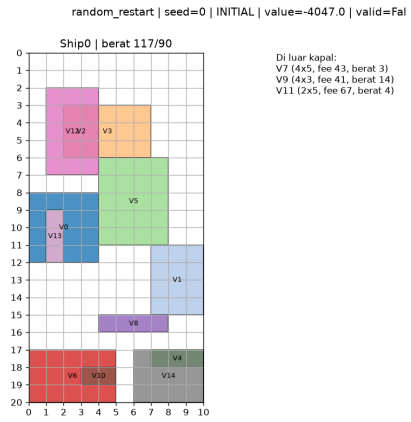
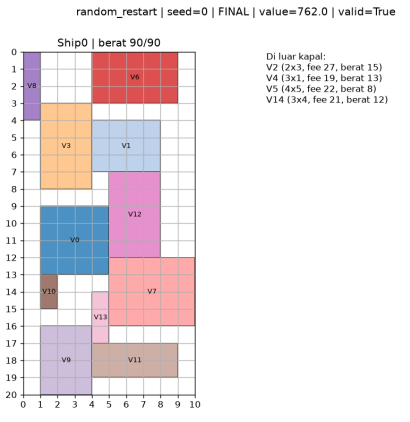
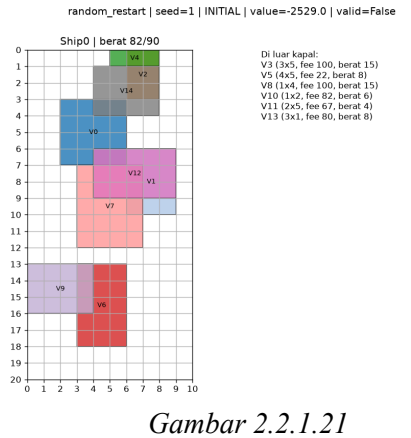
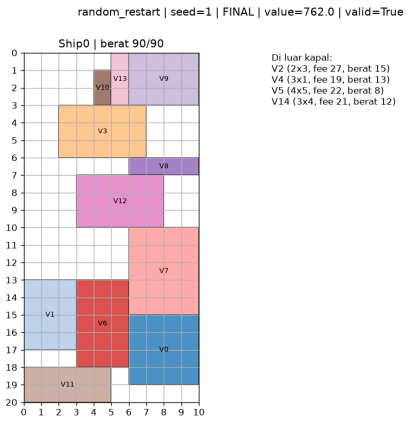
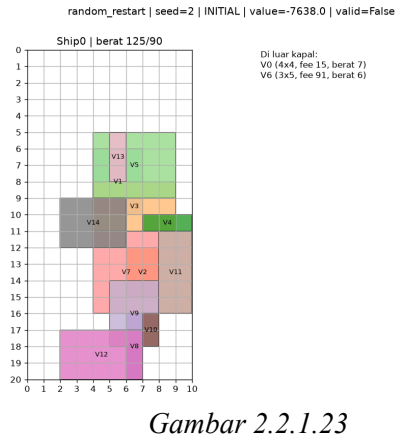
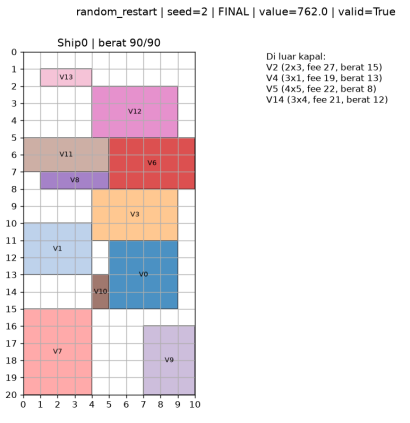
Seed	Iterasi per run (run 0 sampai 10)	Total	Rata-rata
0	8, 8, 8, 9, 8, 5, 6, 8, 9, 6, 9	84	7,6
1	11, 8, 7, 6, 7, 7, 4, 9, 9, 7, 8	83	7,5
2	8, 11, 9, 7, 7, 8, 5, 10, 9, 4, 7	85	7,7

Tabel 2.2.1.3 Visualisasi State Awal dan Akhir Empat Varian *Hill-Climbing*

Varian	Seed	Initial State	Final State
Steepest Ascent	0	 *Gambar 2.2.1.1*	 *Gambar 2.2.1.2*
	1	 *Gambar 2.2.1.3*	 *Gambar 2.2.1.4*
	2	 *Gambar 2.2.1.5*	 *Gambar 2.2.1.6*

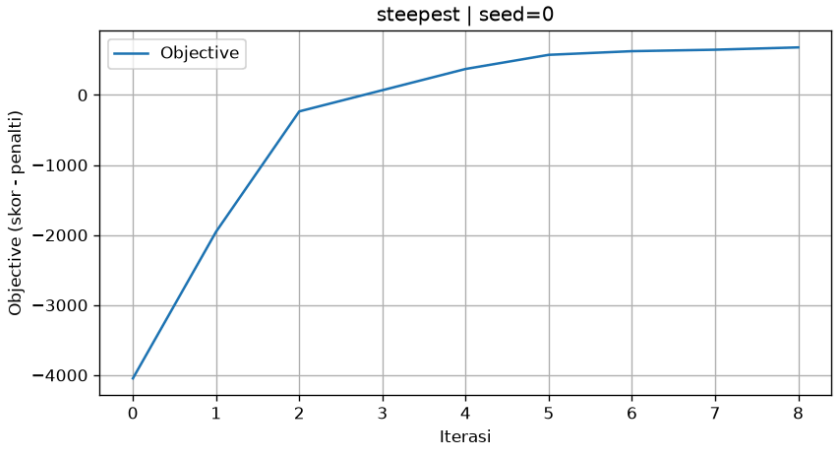
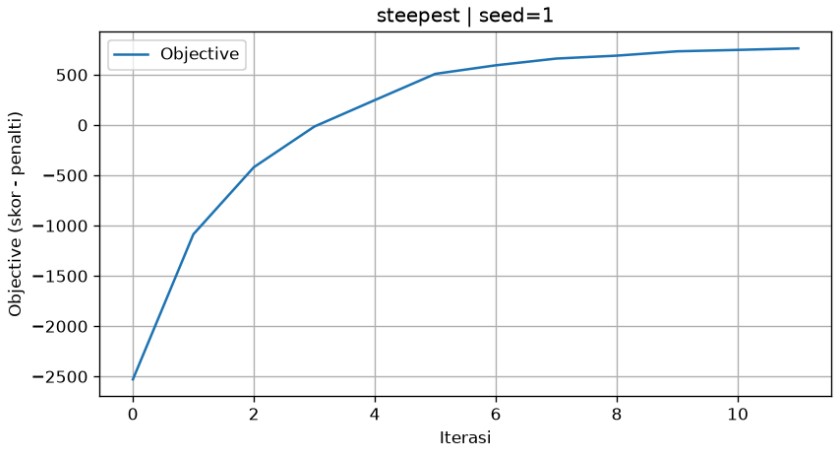
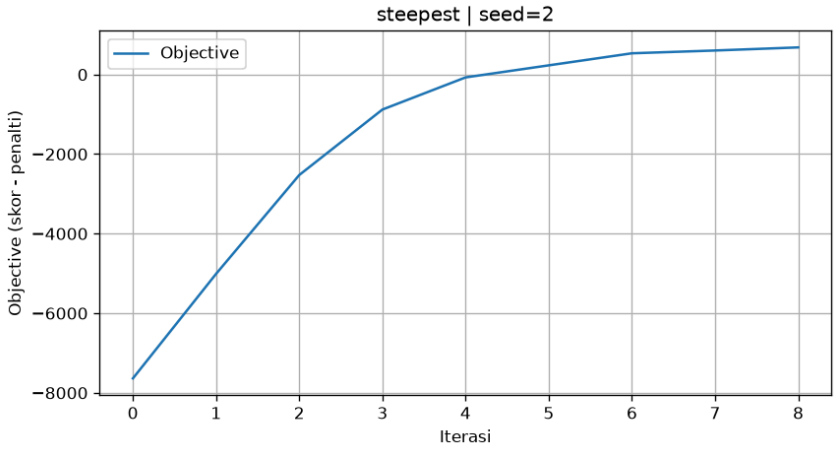
Sideways Move (max 100)	0	 *Gambar 2.2.1.7*	 *Gambar 2.2.1.8*
	1	 *Gambar 2.2.1.9*	 *Gambar 2.2.1.10*
	2	 *Gambar 2.2.1.11*	 *Gambar 2.2.1.12*

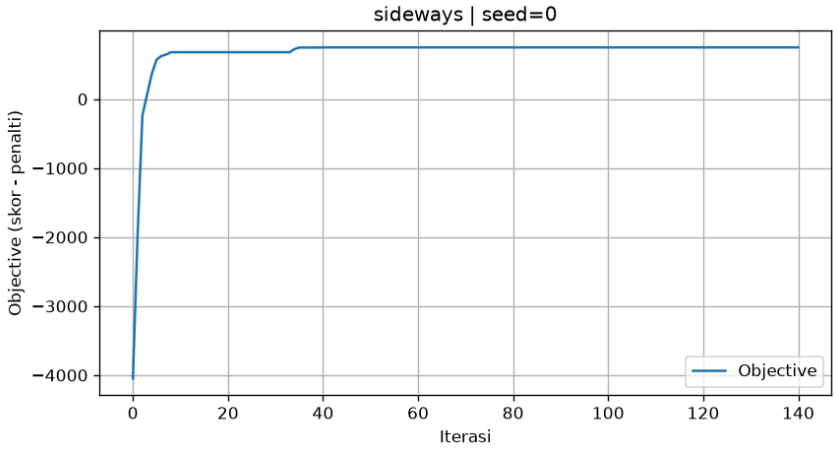
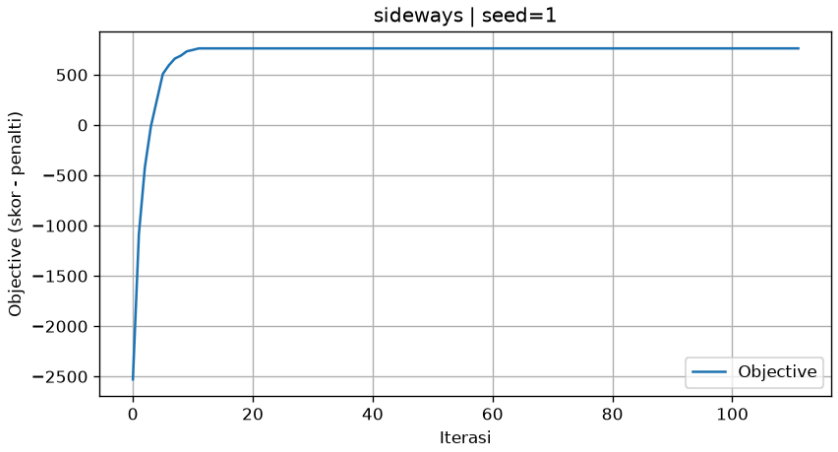
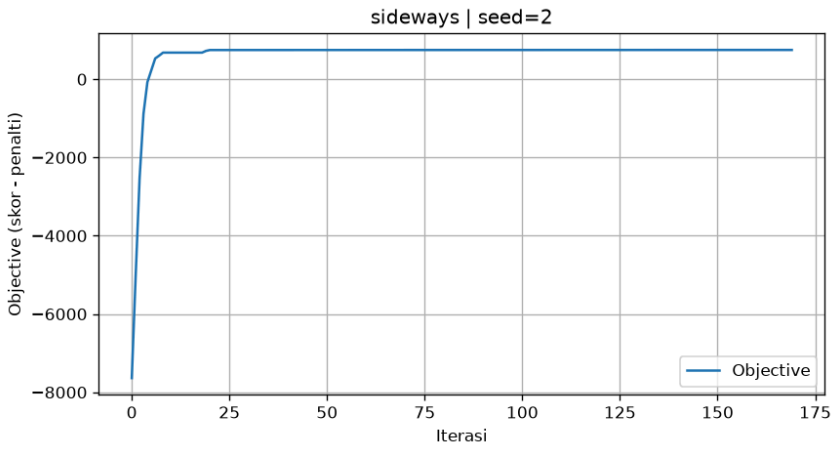
Stochastic	0	 *Gambar 2.2.1.13*	 *Gambar 2.2.1.14*
	1	 *Gambar 2.2.1.15*	 *Gambar 2.2.1.16*
	2	 *Gambar 2.2.1.17*	 *Gambar 2.2.1.18*

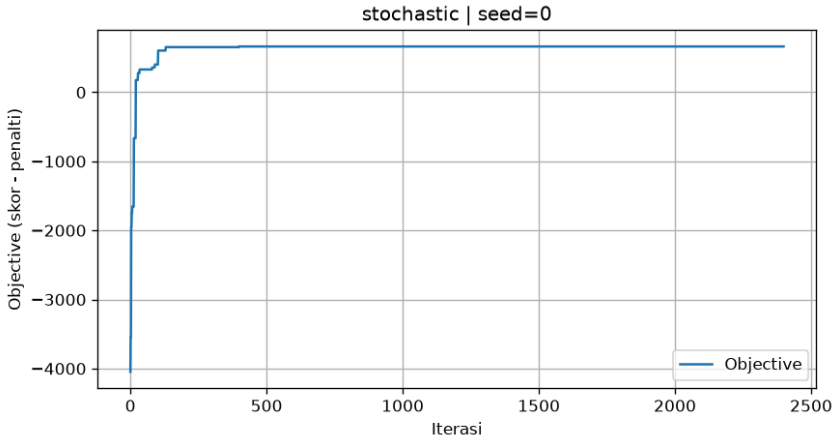
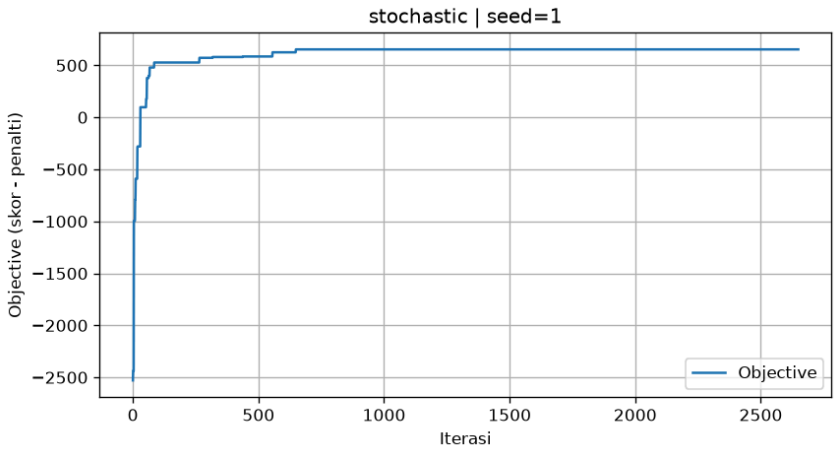
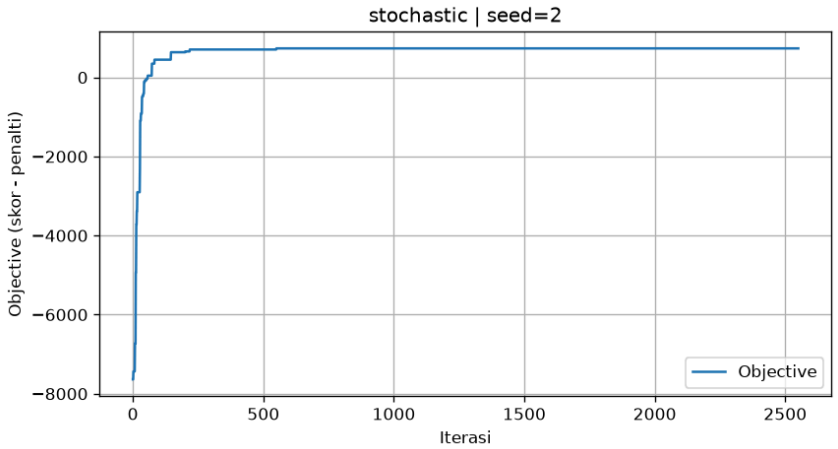
Random Restart (10 restart)	0	 *Gambar 2.2.1.19*	 *Gambar 2.2.1.20*
	1	 *Gambar 2.2.1.21*	 *Gambar 2.2.1.22*
	2	 *Gambar 2.2.1.23*	 *Gambar 2.2.1.24*

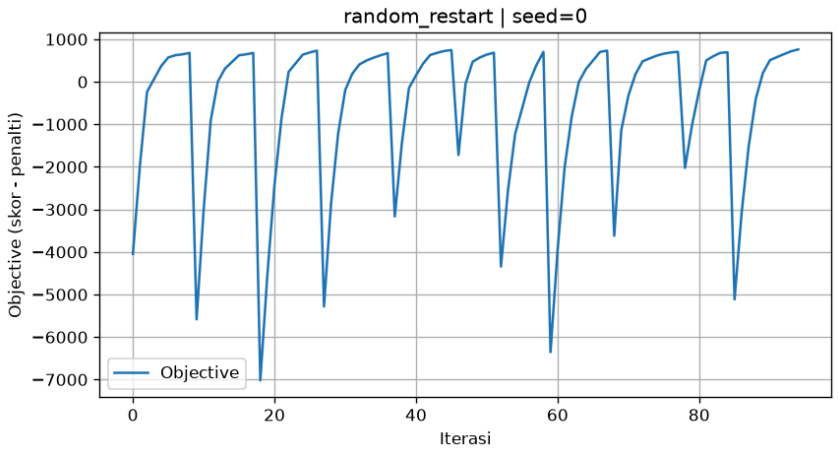
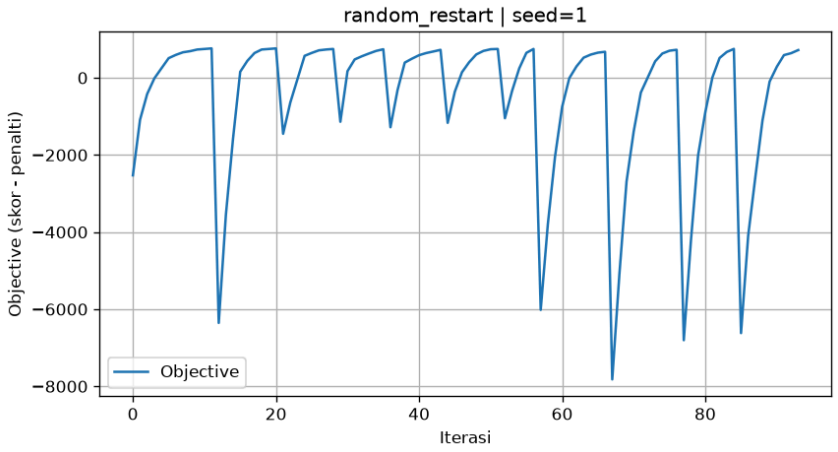
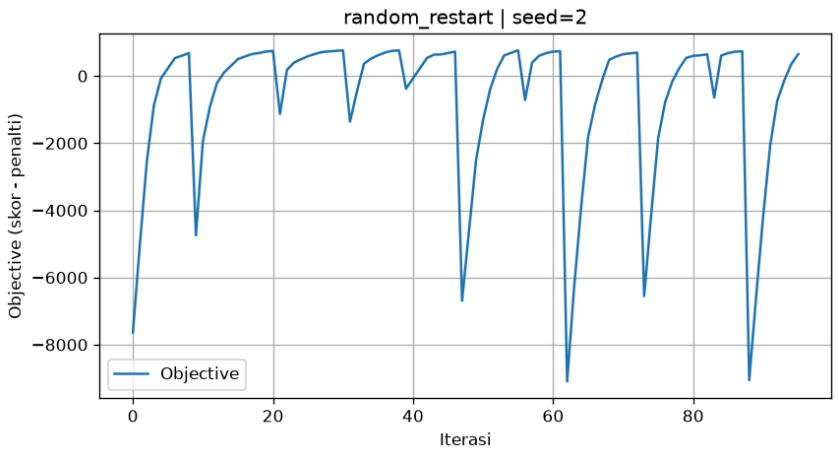
Tabel 2.2.1.4 Kurva Nilai Objective Empat Varian Hill-Climbing

Varian	Seed	Objective Curve

Steepest Ascent	0	 *Gambar 2.2.1.25*
	1	 *Gambar 2.2.1.26*
	2	 *Gambar 2.2.1.27*

Sideways Move (max 100)	0	 *Gambar 2.2.1.28*
	1	 *Gambar 2.2.1.29*
	2	 *Gambar 2.2.1.30*

Stochastic	0	 *Gambar 2.2.1.31*
	1	 *Gambar 2.2.1.32*
	2	 *Gambar 2.2.1.33*

Random Restart (10 restart)	0	 *Gambar 2.2.1.34*
	1	 *Gambar 2.2.1.35*
	2	 *Gambar 2.2.1.36*

1. **Steepest Ascent**

Steepest Ascent berhenti pada 679, 762, dan 681 (rata-rata 707,3 atau 7,2% di bawah optimum) setelah hanya 8 sampai 11 iterasi. Kurva objective naik tajam dari nilai sangat negatif karena pada iterasi-iterasi awal tiap langkah menghilangkan banyak

penalti, lalu mendatar. Algoritma berhenti karena tidak ada satu pun dari ribuan neighbor yang lebih baik. Penyebab utamanya adalah bentuk landscape akibat penalti: satu sel overlap berharga `P = 101`, lebih besar dari tarif kendaraan mana pun (tarif tertinggi 100). Memperbaiki susunan yang sudah padat umumnya harus melalui keadaan overlap sesaat dengan nilai jauh lebih rendah, yang tidak pernah dipilih oleh pendakian yang hanya menerima perbaikan. Hasil 679 dan 681 memuat 10 kendaraan dengan berat 90 dan 89, sehingga kapasitas hampir penuh tetapi kombinasi kendaraannya kurang menguntungkan. Untuk mencapai 762 perlu menukar kendaraan (mengeluarkan satu dan memasukkan yang lain), dan itu hanya mungkin lewat keadaan yang lebih buruk lebih dulu. Hanya seed 1 yang langsung mencapai 762. Kondisi ini adalah local optimum klasik yang hanya dapat dilewati dengan langkah yang menurunkan nilai sementara.

2. **Sideways Move**

   Izin bergerak di dataran meningkatkan hasil menjadi 749, 762, dan 749 (rata-rata 753,3 atau 1,1% di bawah optimum), naik 70, 0, dan 68 poin dari Steepest pada seed yang sama. Dengan `max_sideways = 100`, jumlah iterasi 140, 111, dan 169 menunjukkan bahwa nilai akhir dicapai pada iterasi sekitar 40, 11, dan 69, lalu sisa 100 iterasi hanya berjalan di dataran. Artinya pencarian berhenti karena batas, bukan karena benar-benar tidak ada jalan keluar. Kekurangannya adalah dari segi waktu di mana Sideways Move mengambil 9,3 sampai 13,3 detik (rata-rata 11,7 detik) atau sekitar 15 kali lebih lama daripada Steepest (0,78 detik), karena setiap iterasi tetap mengevaluasi seluruh (sekitar 2.100) neighbor dan iterasinya mencapai 111 sampai 169.

3. **Stochastic**

   Hasilnya 663, 653, dan 733 (rata-rata 683,0 atau 10,4% di bawah optimum), terburuk di antara semua algoritma. Perbaikan terakhir terjadi pada iterasi sekitar 399, 649, dan 550 (iterasi akhir dikurangi patience 2.000); batas `max_iter 20.000` tidak pernah tercapai. Algoritma ini paling murah (0,10 sampai 0,14 detik, sekitar 6 kali lebih cepat dari Steepest) karena hanya mengevaluasi satu neighbor per iterasi. Secara rata-rata Stochastic tidak lebih baik dari Steepest (683,0 vs 707,3). Stochastic hanya unggul pada seed 2 (733 vs 681) dan kalah pada seed 0 (663 vs 679) dan seed 1 (653 vs 762). Kami menduga sebagian besar neighbor acak adalah langkah yang tidak memperbaiki, sehingga pencarian cenderung terjebak di optimum lokal. Pencarian berhenti pada iterasi sekitar 399, 649, dan 550 padahal patience 2.000 sehingga yang

membatasi hasil adalah sifat pencariannya (hanya menerima perbaikan dari neighbor acak).

4. **Random Restart**

   Dengan 11 run steepest, ketiga seed mencapai 762 (rata-rata 762,0, gap 0%), terbaik di antara semua algoritma. Pada seed 0 dan 2, hasil restart (762) lebih tinggi dari hasil satu kali Steepest dari initial state yang sama (679 dan 681), sedangkan pada seed 1 satu kali Steepest sudah mencapai 762. Hal ini menunjukkan bahwa basin of attraction menuju 762 ada dan dapat ditemukan dengan sampling titik awal. Biayanya sekitar 8 kali Steepest (6,5 sampai 6,7 detik), sedikit di bawah jumlah run (11) karena waktu tiap run steepest tidak seragam.

Rentang hasil akhir (maks - min) adalah 83 untuk Steepest, 80 untuk Stochastic, 13 untuk Sideways, dan 0 untuk Random Restart. Varian yang dapat meloloskan diri dari plateau atau local optimum (sideways dan restart) tidak hanya lebih tinggi tetapi juga lebih konsisten antar seed.

2.2.2 *Simulated Annealing*

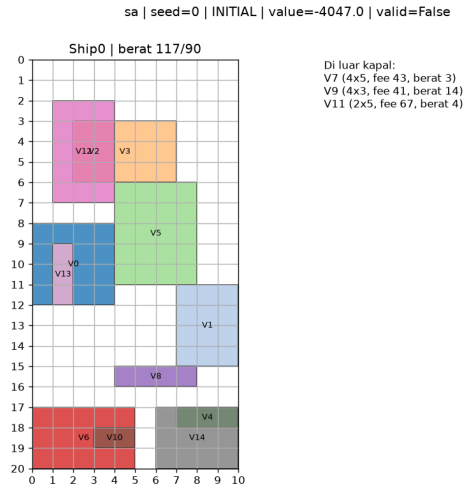
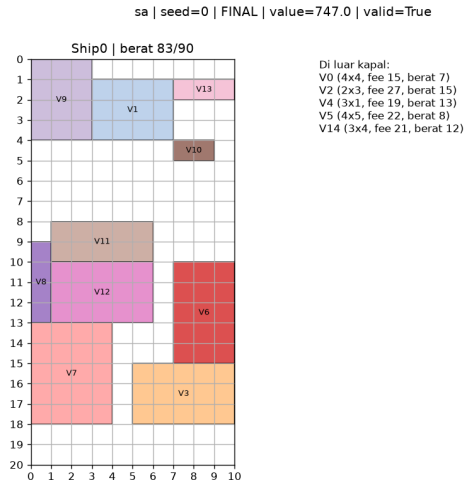
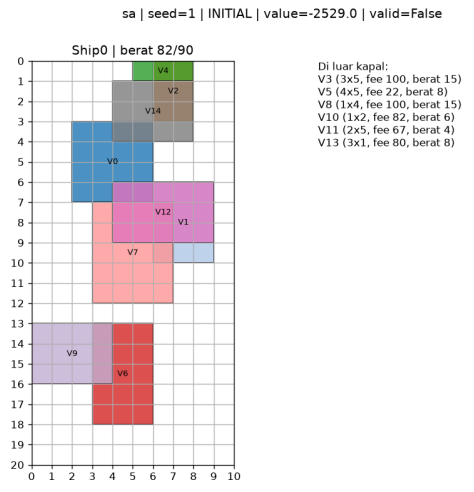
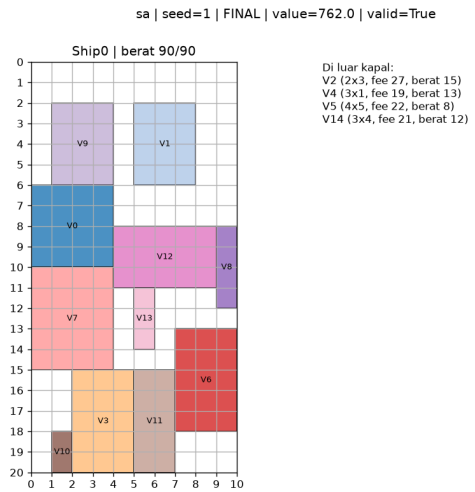
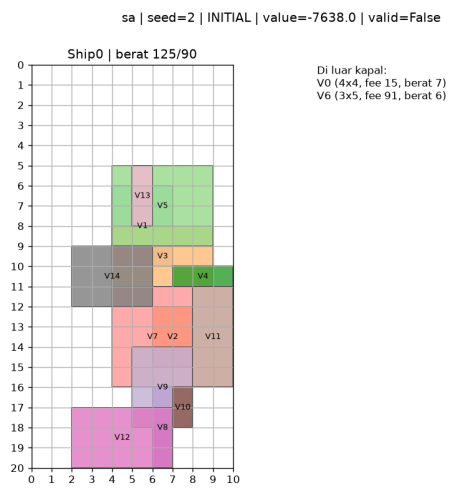
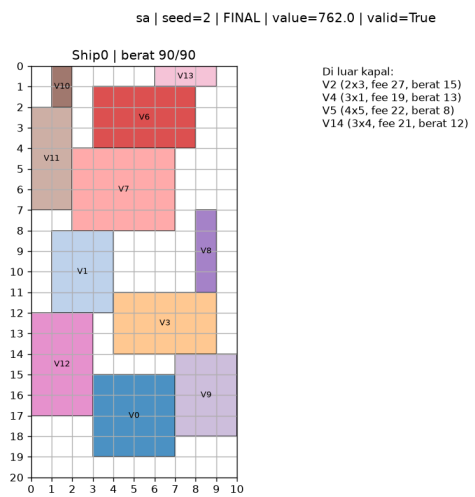
Hasil tiga run (T0 = 60, α = 0,999, T_min = 0,01).

Tabel 2.2.2.1 Hasil Pengujian *Simulated Annealing* pada Case1

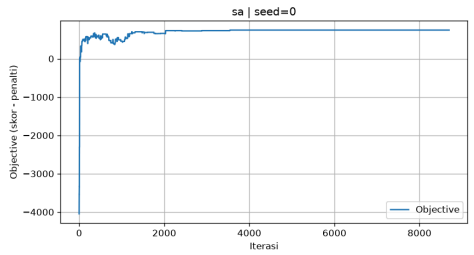
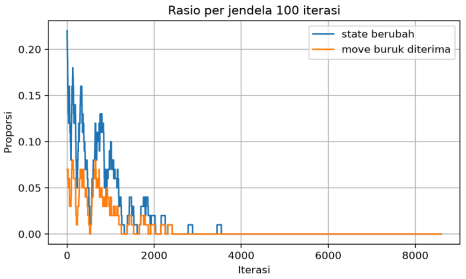
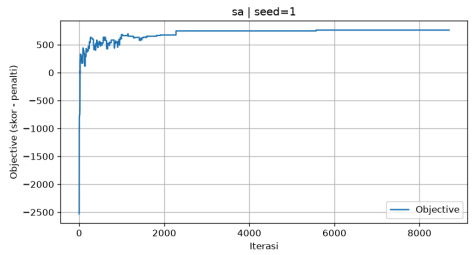
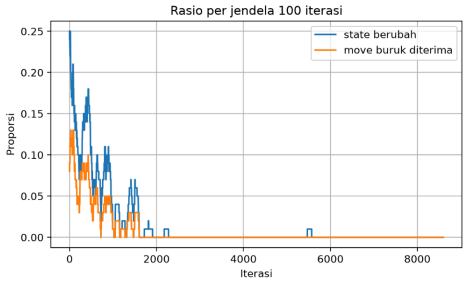
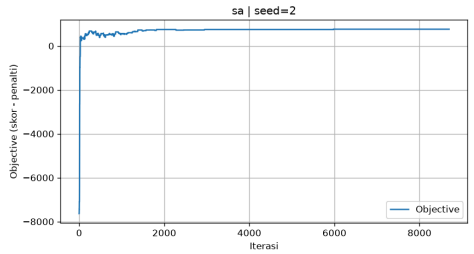
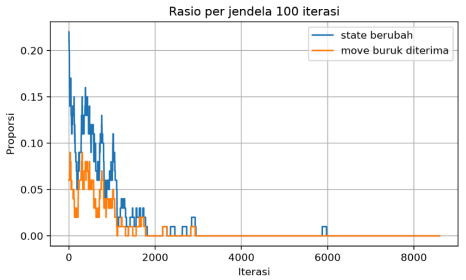
Seed	Initial	Final	Gap ke 762	Iterasi	Durasi total (s)	Langkah memburuk ditolak (stuck)	Hit check LO	Dimuat / berat
0	-4.047	747	2,0%	8.696	6,64	5.789 (66,6%)	64 / 173 (37,0%)	10 / 83
1	-2.529	762	0%	8.696	8,17	5.904 (67,9%)	90 / 173 (52,0%)	11 / 90
2	-7.638	762	0%	8.696	7,77	6.054 (69,6%)	75 / 173 (43,4%)	11 / 90

Durasi total mencakup 173 pengecekan local optimum per run (sekitar 95% dari total); durasi pencarian murni hanya sekitar 0,35 detik. Hit check LO adalah jumlah pengecekan yang mendapati state saat itu local optimum, dari 173 pengecekan per run (tiap 50 iterasi).

Tabel 2.2.2.2 Visualisasi State Awal dan Akhir *Simulated Annealing*

Seed	Initial	Final
0	 *Gambar 2.2.2.1*	 *Gambar 2.2.2.2*
1	 *Gambar 2.2.2.3*	 *Gambar 2.2.2.4*
2	 *Gambar 2.2.2.5*	 *Gambar 2.2.2.6*

Tabel 2.2.2.3 Kurva Nilai *Objective* dan Rasio Penerimaan *Simulated Annealing*

Seed	Objective Curve	Rasio per jendela 100 iterasi
0	 *Gambar 2.2.2.7*	 *Gambar 2.2.2.8*
1	 *Gambar 2.2.2.9*	 *Gambar 2.2.2.10*
2	 *Gambar 2.2.2.11*	 *Gambar 2.2.2.12*

1. **Seberapa dekat dengan optimum global**

   Dua dari tiga run (seed 1 dan 2) mencapai 762, yaitu optimum global, sedangkan seed 0 berhenti di 747 (rata-rata 757,0 atau 0,7% di bawah optimum, simpangan baku 8,7). Selisih 15 poin dan 7 satuan berat pada seed 0 (10 kendaraan, berat 83) sesuai dengan V0 (tarif 15, berat 7) yang tidak dimuat, padahal kapasitas masih cukup. Jumlah iterasi tetap 8.696 karena kondisi berhenti ditentukan oleh jadwal suhu. Nilai ini sedikit di atas Sideways (753,3), tetapi di bawah Random Restart (762,0) yang konsisten di ketiga seed.

2. **Perilaku kurva**

Kurva objective memperlihatkan tiga fase. Pada awal pencarian, objective naik dari nilai sangat negatif ke kisaran 500 pada iterasi ke-100. Penalti dihilangkan lebih dulu karena langkah yang menciptakan overlap bernilai sangat buruk, sehingga peluang penerimaannya, e^(-101/T), kecil (sekitar 0,19 pada $T_0 = 60$ dan di bawah 0,01 saat T ≈ 22). Kemudian, pada iterasi 500 sampai 2.000, nilai berfluktuasi di rentang sekitar 380 sampai 710 dan sering turun di bawah hasil Hill-Climbing. Penyebabnya, suhu masih tinggi (T ≈ 36 pada iterasi 500, ≈ 22 pada iterasi 1.000, dan ≈ 8 pada iterasi 2.000), sehingga langkah yang menurunkan puluhan poin masih sering diterima. Sebagai contoh, penurunan 15 poin pada T = 36 diterima dengan peluang sekitar 0,66. Fluktuasi ini adalah harga eksplorasi yang memungkinkan SA keluar dari basin yang menjebak Steepest pada seed 0 dan 2. Nilai 762 pertama kali dicapai pada iterasi 5569 (seed 1) dan 5976 (seed 2). Seed 0 tidak pernah mencapainya dan berhenti di 747 (selisih 2,0%). Pada fase akhir kurva mendatar karena suhu sudah terlalu rendah untuk meninggalkan nilai tersebut. Penerimaan langkah memburuk praktis berhenti sekitar iterasi 2.500 (lihat grafik rasio), dan nilai objective terakhir berubah pada iterasi 3.544 (seed 0), 5.569 (seed 1), dan 5.976 (seed 2). Proporsi iterasi yang berada di 762 sebesar 36% (seed 1) dan 31% (seed 2).

3. **Plot e^(ΔE/T).**

Peluang penerimaan langkah memburuk bergantung pada suhu dan besar penurunan. Pada awal pencarian peluang untuk penurunan puluhan poin masih berkisar di atas 0,5. Seiring T turun secara geometrik, peluang untuk penurunan yang sama menyusut tajam menuju 0 (misalnya e^(-15/0,5) ≈ 10^-13). Langkah dengan ΔE = 0 (sideways) selalu berpeluang 1 dan menyumbang 32,1%, 30,7%, dan 29,0% dari seluruh langkah non-meningkat pada seed 0, 1, dan 2. Rata-rata peluang seluruh langkah non-meningkat adalah 0,327; 0,315; dan 0,297 (rata-rata 0,313), sedangkan mediannya hanya sekitar 10^-26 sampai 10^-35. Distribusinya bimodal (satu kelompok di 1 dan satu kelompok mendekati 0), sehingga rata-rata ini terdorong naik oleh langkah ΔE = 0 dan tidak mencerminkan peluang penerimaan langkah yang benar-benar memburuk, yang pada fase akhir hampir nol.

4. **Frekuensi stuck**

Pada implementasi kami, stuck menghitung langkah memburuk yang ditolak, yaitu iterasi saat SA bertahan di state yang sama meskipun ada kandidat. Angkanya 5.789 sampai 6.054 dari 8.696 iterasi (66,6 sampai 69,6%, rata-rata sekitar 68%). Sebagian besar terjadi pada fase akhir ketika suhu rendah dan SA praktis berperilaku seperti

Hill-Climbing yang sudah berada di local optimum. Pada fase suhu tinggi hampir semua langkah memburuk diterima sehingga hampir tidak ada penolakan. Metrik ini adalah pendekatan terhadap "frekuensi stuck di local optima", bukan deteksi local optimum yang eksak.

5. **Hit check local optimum**

   Pada pengecekan tiap 50 iterasi, SA berada di local optimum pada 64, 90, dan 75 dari 173 pengecekan (37,0%, 52,0%, dan 43,4%; rata-rata 44,1%). Dari hit tersebut, 14, 28, dan 21 (21,9%, 31,1%, dan 28,0%) terjadi pada local optimum yang nilainya di bawah hasil akhir, artinya SA sempat terjebak di local optimum yang lebih rendah lalu berhasil meninggalkannya. Hal ini mendukung peran penerimaan langkah memburuk. Angka 'di bawah final' dihitung dari `extras["lo_log"]` pada `result.json`, yaitu jumlah pengecekan yang nilainya di bawah nilai akhir.

6. **Waktu**

   Durasi total yang tercatat 6,6 sampai 8,2 detik (rata-rata 7,5 detik), tetapi sekitar 95% di antaranya adalah waktu 173 pengecekan local optimum, yang masing-masing mengevaluasi seluruh sekitar 2.100 neighbor. Waktu pencarian murni hanya 0,34 sampai 0,36 detik (rata-rata 0,35 detik), sekitar 2 kali lebih cepat dari Steepest (0,78 detik), sekitar 3 kali lebih lama dari Stochastic (0,12 detik), dan sekitar 19 kali lebih cepat dari Random Restart (6,63 detik). Hal ini sesuai dengan jumlah evaluasi SA yang hanya sekitar 8.700 (satu neighbor per iterasi)

2.2.3 *Genetic Algorithm*

1. Percobaan A: populasi dikontrol (50), jumlah generasi bervariasi

Tabel 2.2.3.1 Hasil Pengujian *Genetic Algorithm* dengan Variasi Jumlah Generasi pada Populasi 50

Generasi	Seed	Initial	Final	Gap ke 762	Durasi (s)
50	0	-4.047	595	21,9%	0,14
	1	-2.529	682	10,5%	0,37
	2	-7.638	640	16,0%	0,26
150	0	-4.047	733	3,8%	0,83
	1	-2.529	725	4,9%	0,82

	2	-7.638	704	7,6%	0,66
400	0	-4.047	747	2,0%	2,14
	1	-2.529	733	3,8%	2,15
	2	-7.638	719	5,6%	2,34

2. Percobaan B: generasi dikontrol (200), jumlah populasi bervariasi

Tabel 2.2.3.2 Hasil Pengujian *Genetic Algorithm* dengan Variasi Ukuran Populasi pada 200 Generasi

Populasi	Seed	Initial	Final	Gap ke 762	Durasi (s)
20	0	-4.047	682	10,5%	0,14
	1	-2.529	747	2,0%	0,18
	2	-7.638	660	13,4%	0,19
50	0	-4.047	733	3,8%	0,63
	1	-2.529	725	4,9%	0,55
	2	-7.638	704	7,6%	1,14
100	0	-4.047	705	7,5%	2,76
	1	-2.529	762	0%	2,68
	2	-7.638	733	3,8%	2,67

3. Ringkasan per konfigurasi.

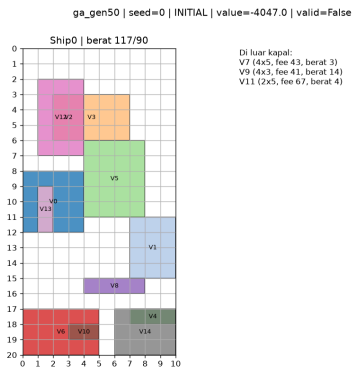
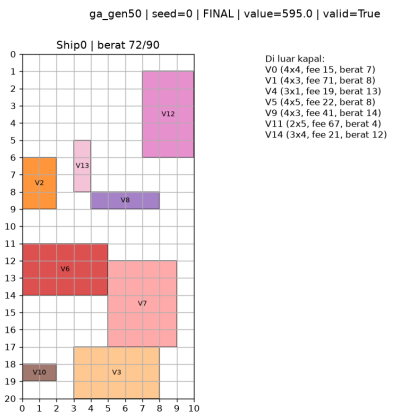
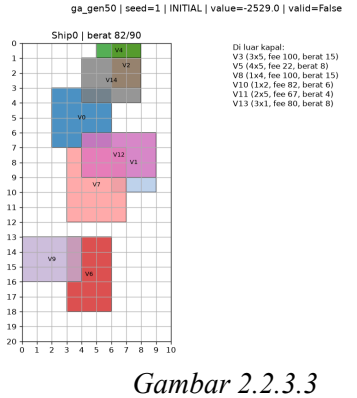
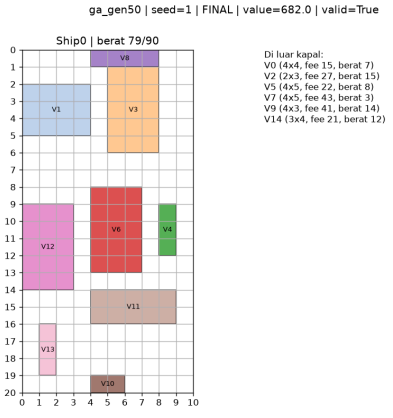
Tabel 2.2.3.3 Ringkasan Hasil Pengujian Seluruh Konfigurasi *Genetic Algorithm*

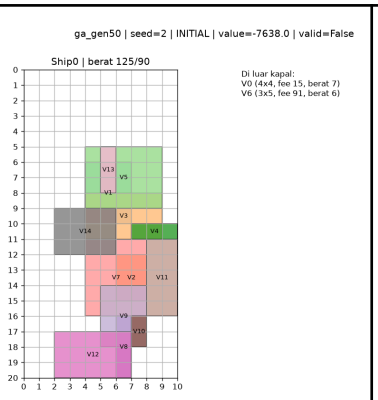
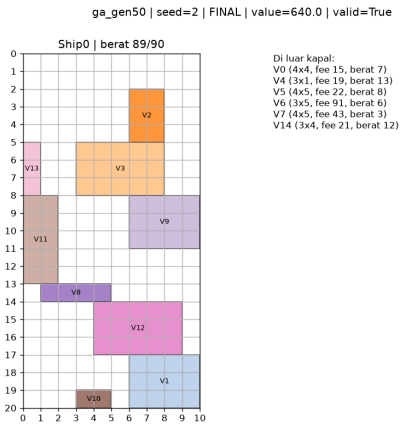
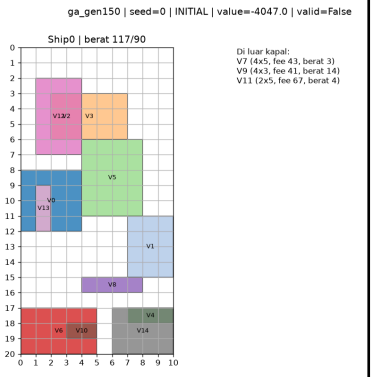
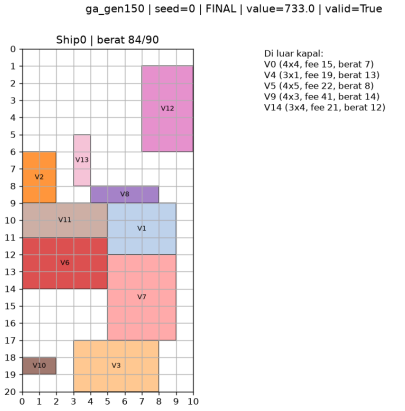
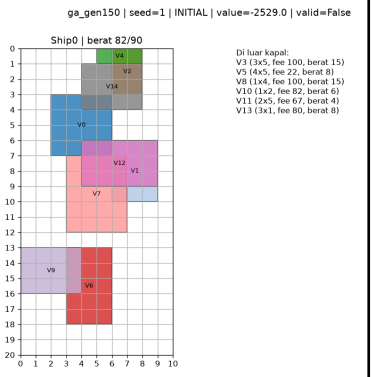
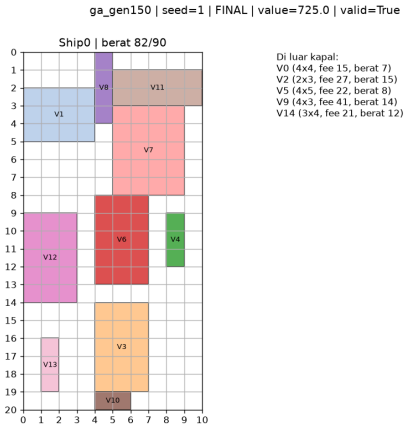
Konfigurasi	Rata-rata final	Simpangan baku	Min – Maks	Gap rata-rata ke 762	Rata-rata durasi (s)
Pop 50, 50 generasi	639,0	43,5	595 – 682	16,1%	0,26
Pop 50, 150 generasi	720,7	15,0	704 – 733	5,4%	0,77
Pop 50, 200 generasi	720,7	15,0	704 – 733	5,4%	0,77

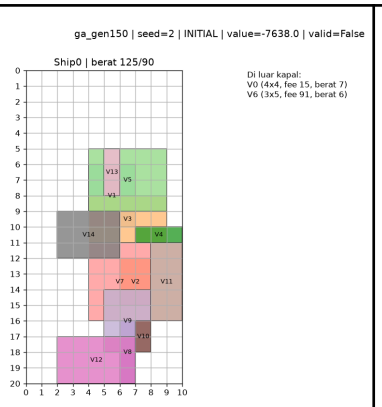
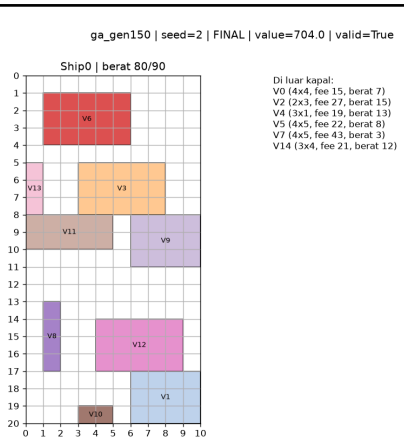
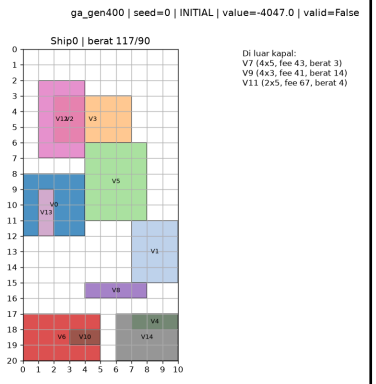
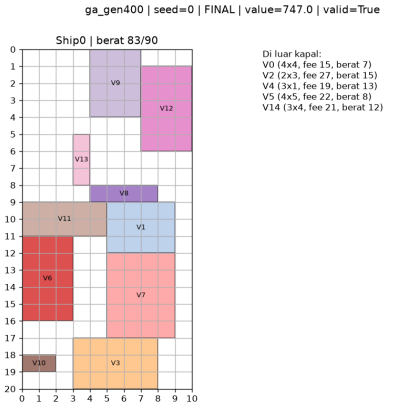
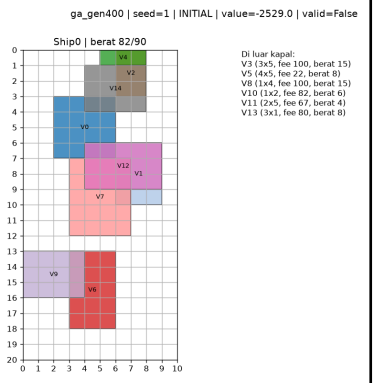
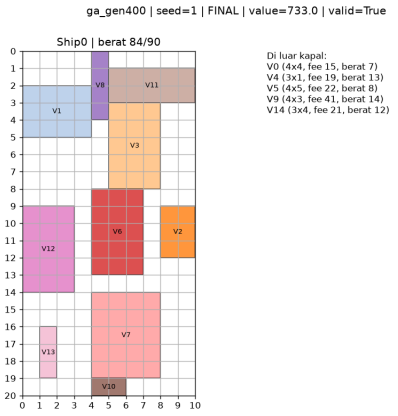
Pop 50, 400 generasi	733,0	14,0	719 – 747	3,8%	2,21
Pop 20, 200 generasi	696,3	45,2	660 – 747	8,6%	0,17
Pop 100, 200 generasi	733,3	28,5	705 – 762	3,8%	2,70

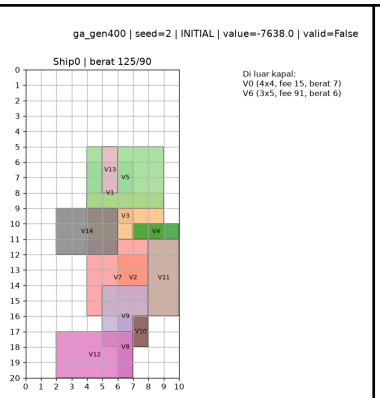
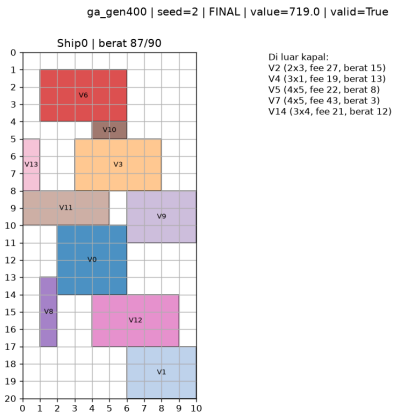
Seluruh hasil akhir valid dan solusi cadangan (all-out fallback) tidak pernah dipakai.

Tabel 2.2.3.4 Visualisasi State Awal dan Akhir *Genetic Algorithm* dengan Variasi Jumlah Generasi

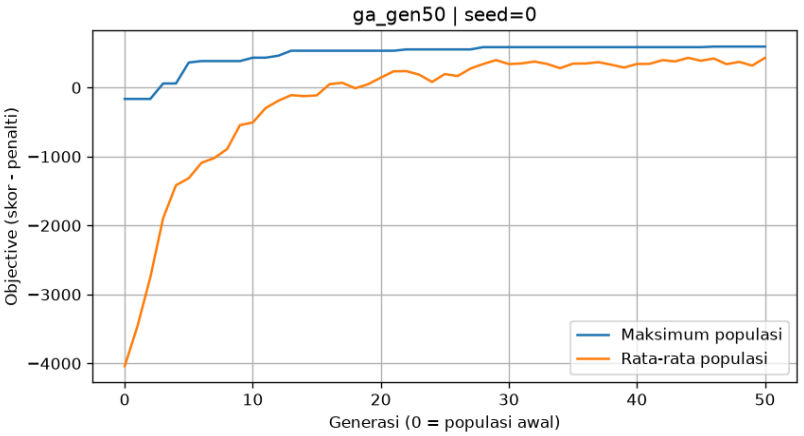
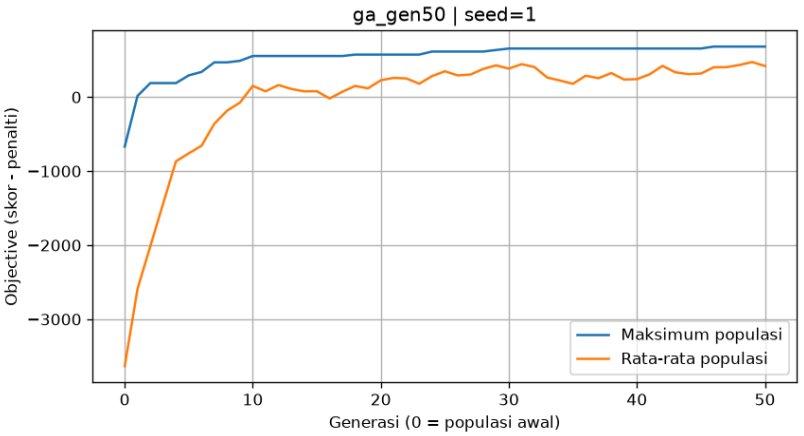
Generasi	Seed	Initial	Final
50	0	 *Gambar 2.2.3.1*	 *Gambar 2.2.3.2*
	1	 *Gambar 2.2.3.3*	 *Gambar 2.2.3.4*

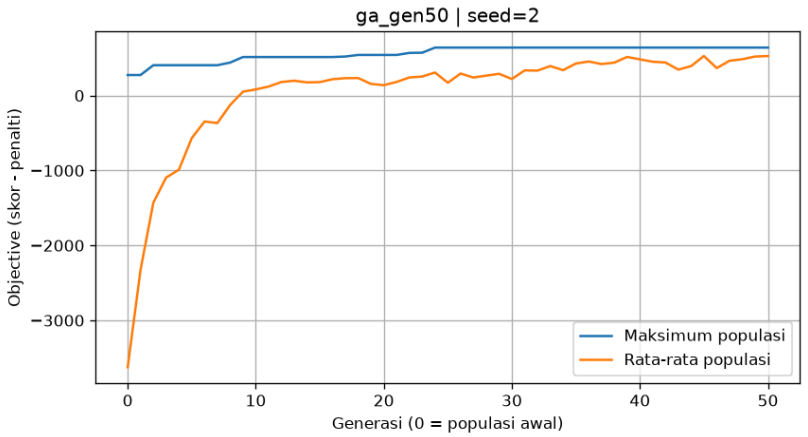
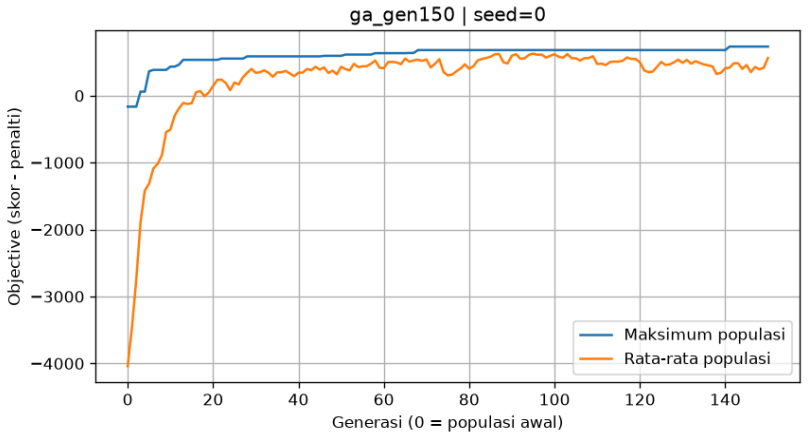
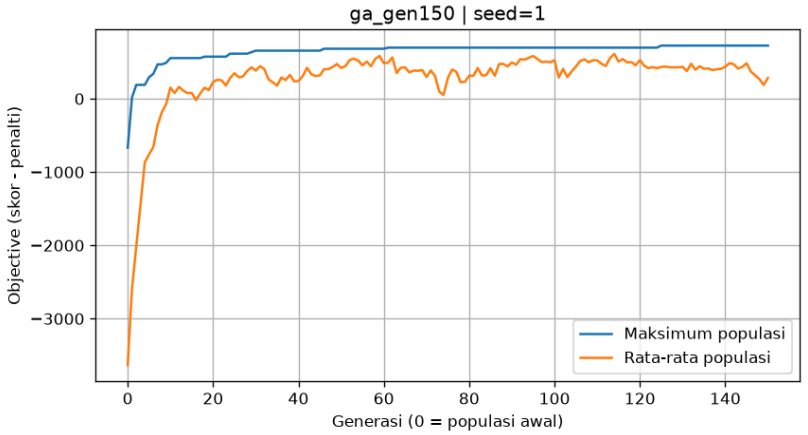
	2	 *Gambar 2.2.3.5*	 *Gambar 2.2.3.6*
150	0	 *Gambar 2.2.3.7*	 *Gambar 2.2.3.8*
	1	 *Gambar 2.2.3.9*	 *Gambar 2.2.3.10*

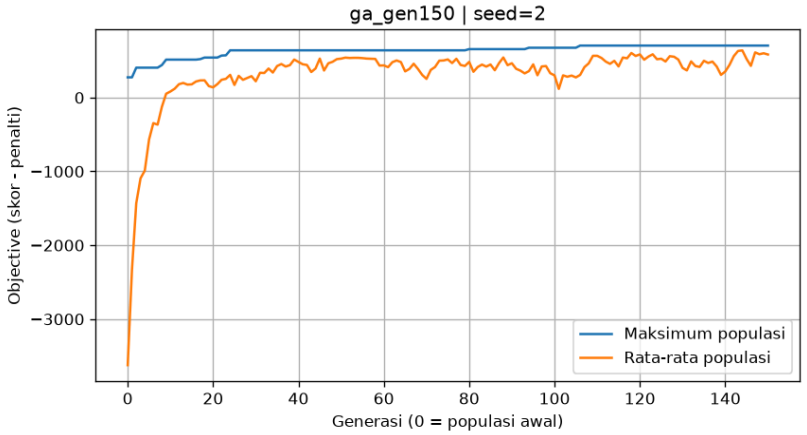
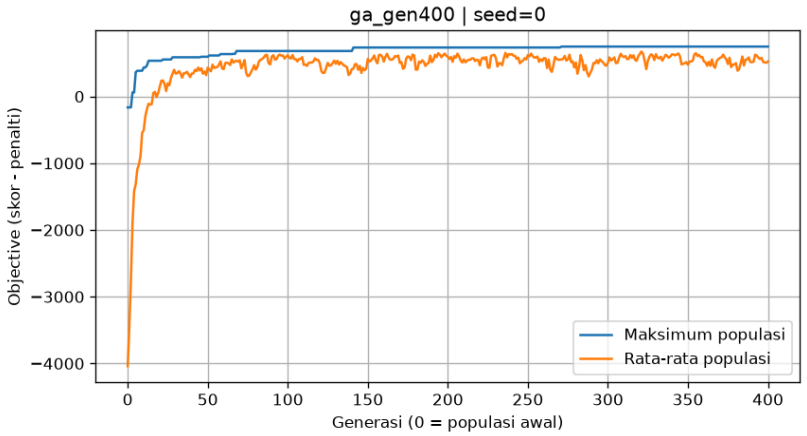
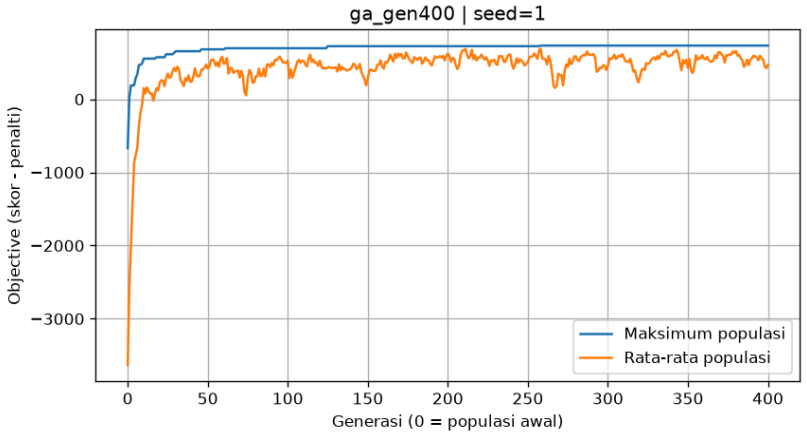
	2	 *Gambar 2.2.3.11*	 *Gambar 2.2.3.12*
400	0	 *Gambar 2.2.3.13*	 *Gambar 2.2.3.14*
	1	 *Gambar 2.2.3.15*	 *Gambar 2.2.3.16*

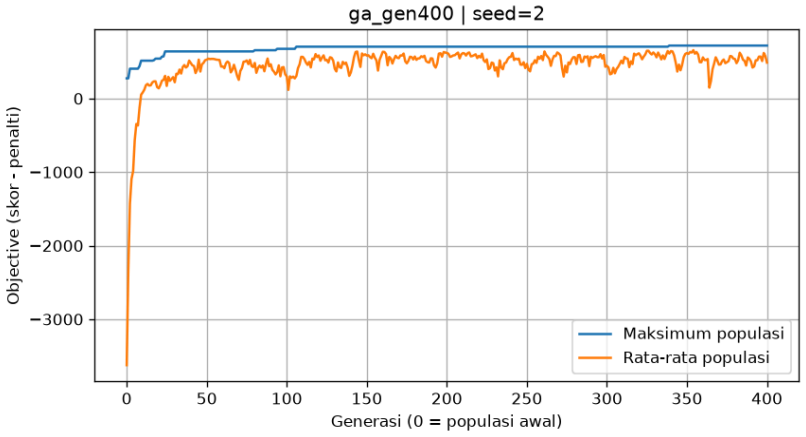
	2	 *Gambar 2.2.3.17*	 *Gambar 2.2.3.18*

Tabel 2.2.3.5 Kurva Fitness Maksimum dan Rata-Rata Genetic Algorithm dengan Variasi Jumlah Generasi

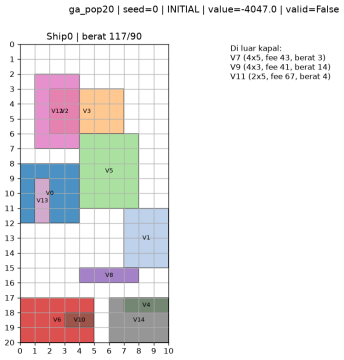
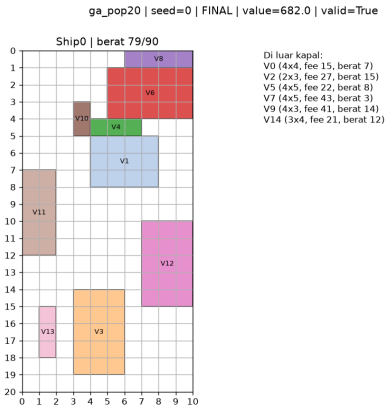
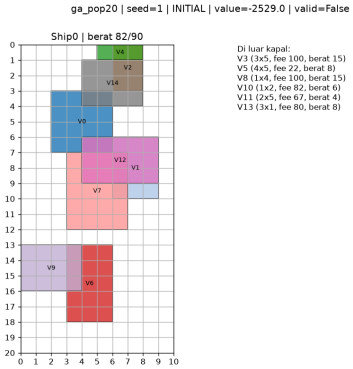
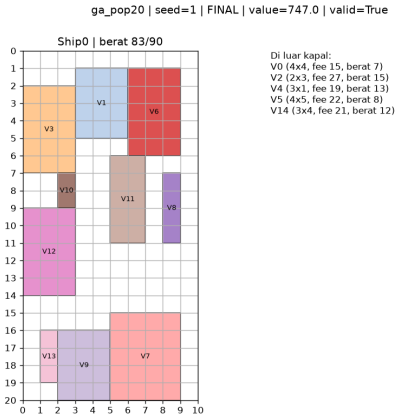
Generasi	Seed	Objective Curve
50	0	 *Gambar 2.2.3.19*
	1	 *Gambar 2.2.3.20*

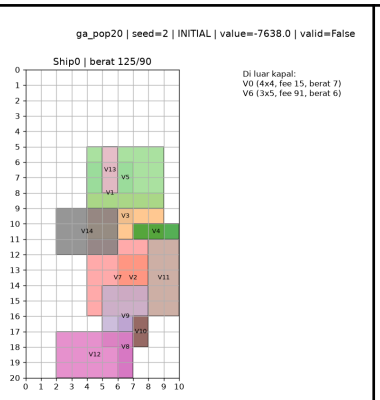
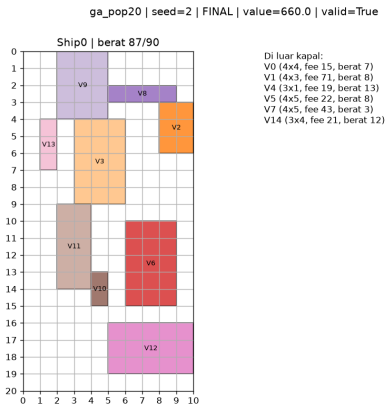
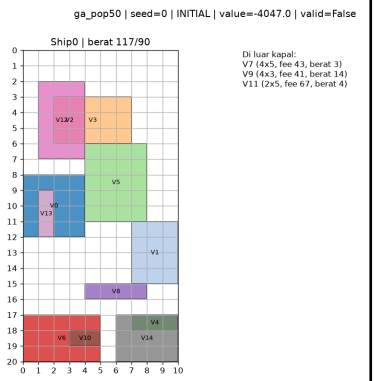
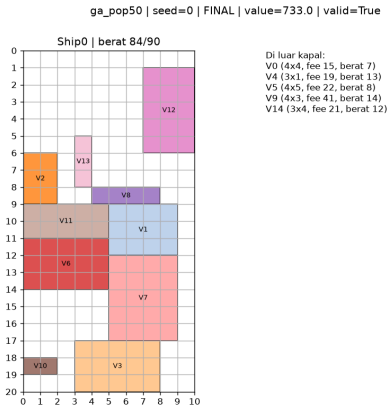
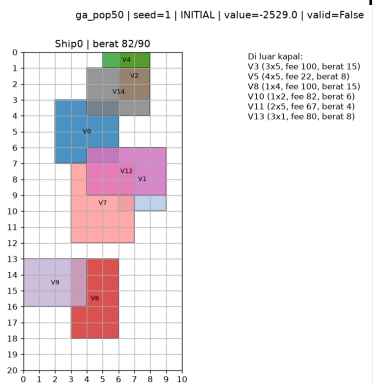
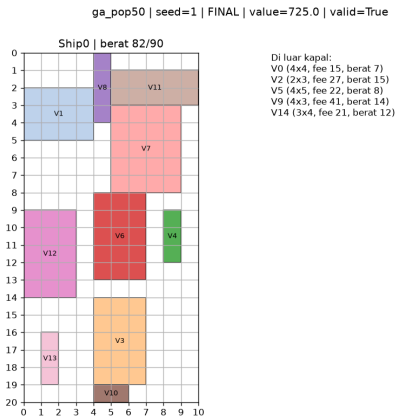
	2	 *Gambar 2.2.3.21*
150	0	 *Gambar 2.2.3.22*
	1	 *Gambar 2.2.3.23*

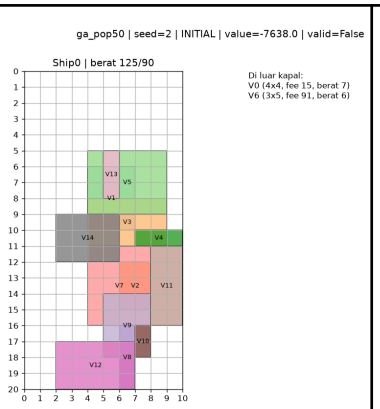
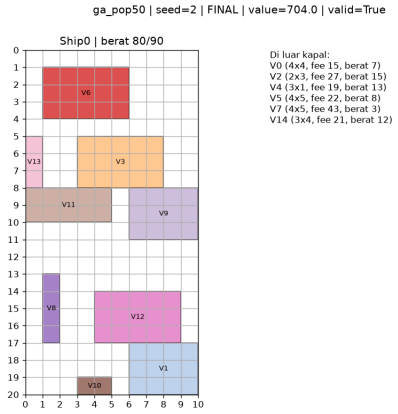
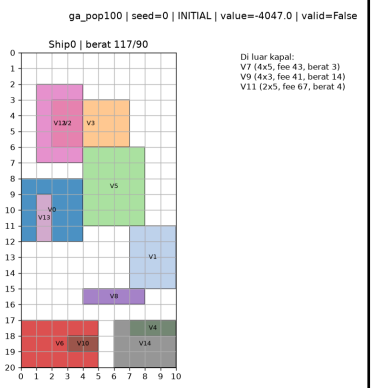
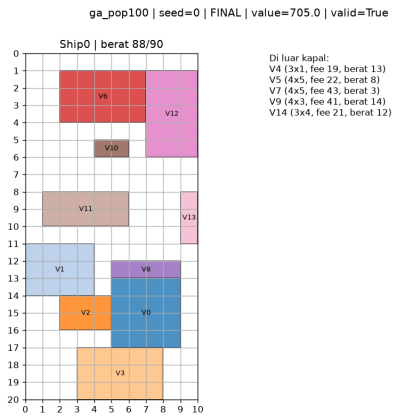
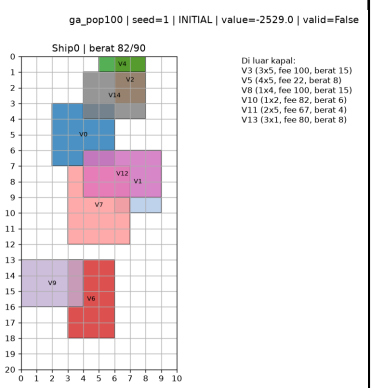
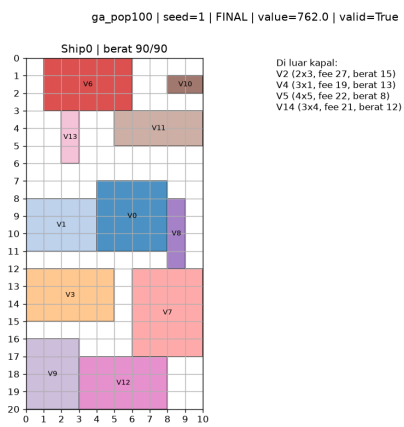
	2	 *Gambar 2.2.3.24*
400	0	 *Gambar 2.2.3.25*
	1	 *Gambar 2.2.3.26*

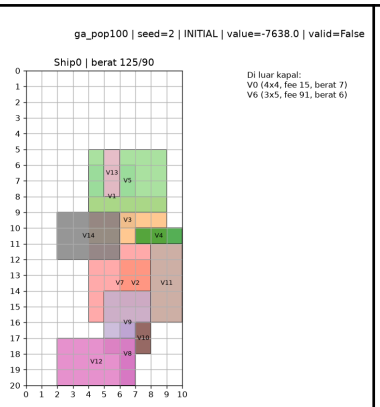
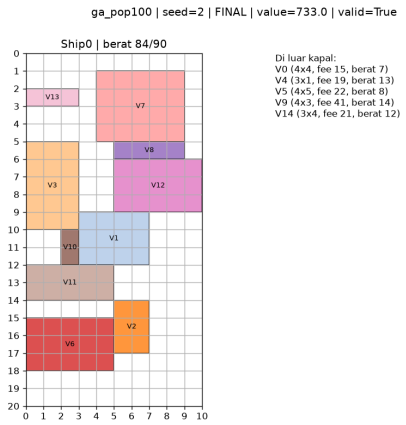
	2	 *Gambar 2.2.3.27*

Tabel 2.2.3.6 Visualisasi State Awal dan Akhir *Genetic Algorithm* dengan Variasi Ukuran Populasi

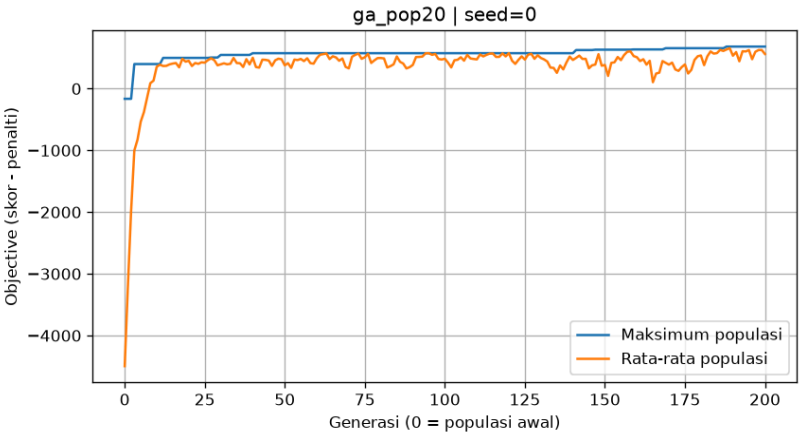
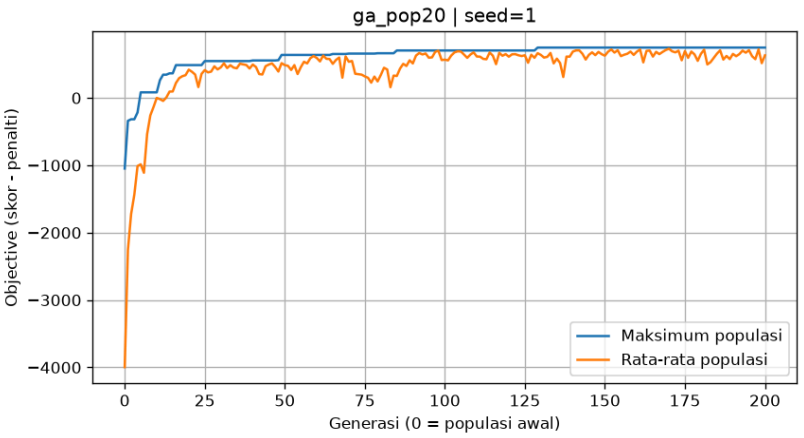
Populasi	Seed	Initial	Final
20	0	 *Gambar 2.2.3.28*	 *Gambar 2.2.3.29*
	1	 *Gambar 2.2.3.30*	 *Gambar 2.2.3.31*

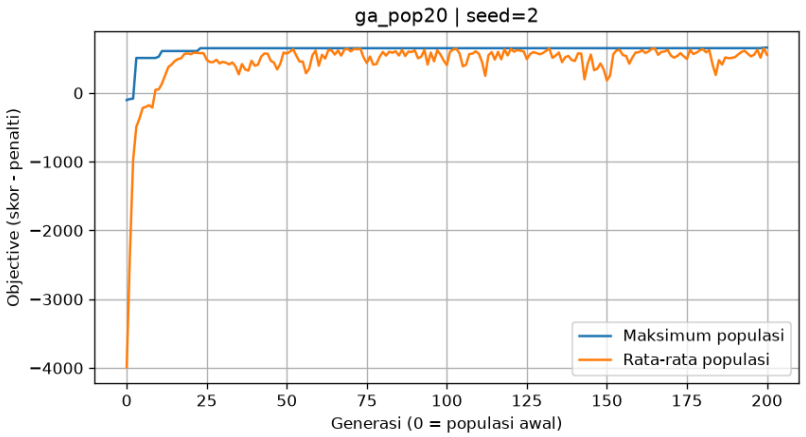
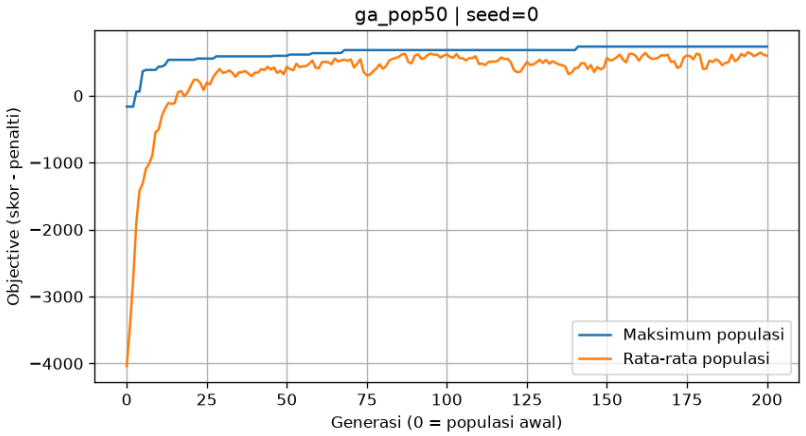
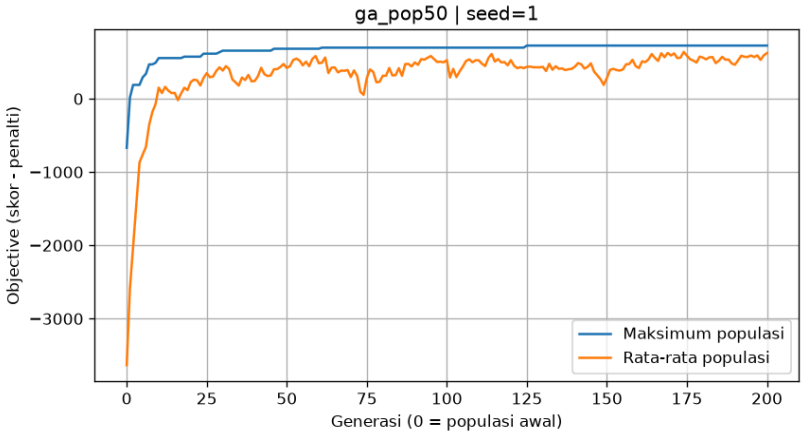
	2	 *Gambar 2.2.3.32*	 *Gambar 2.2.3.33*
50	0	 *Gambar 2.2.3.34*	 *Gambar 2.2.3.35*
	1	 *Gambar 2.2.3.36*	 *Gambar 2.2.3.37*

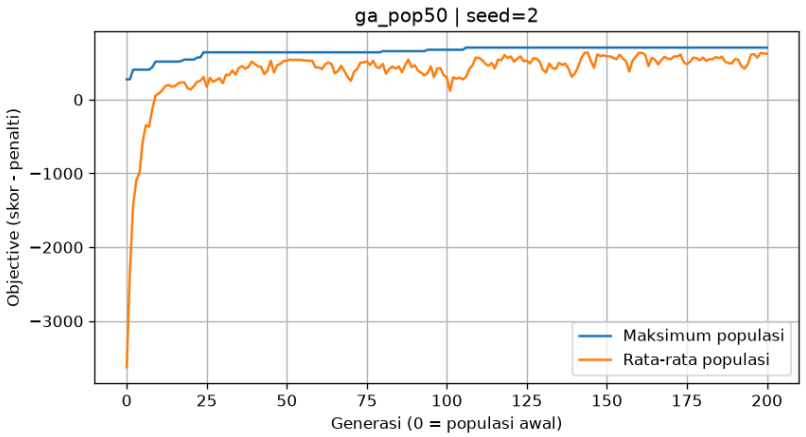
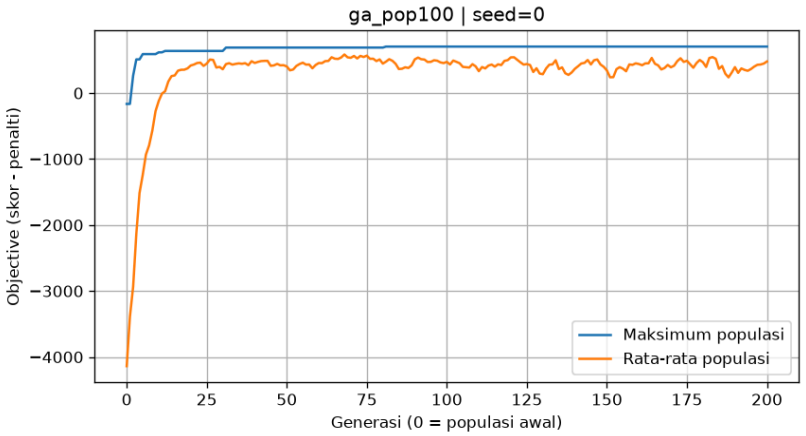
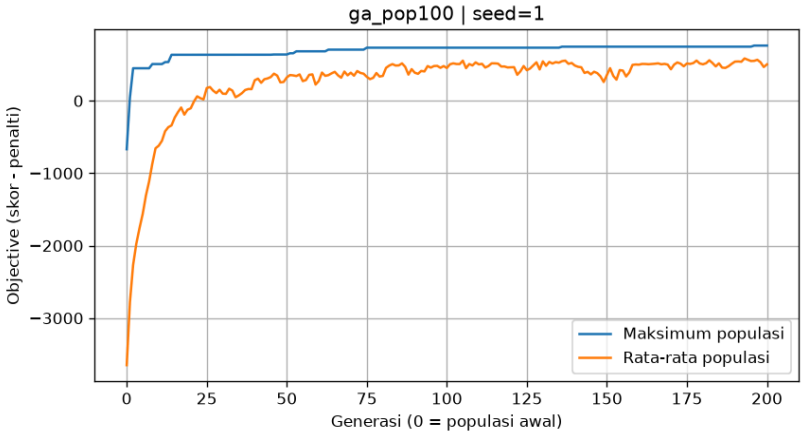
	2	 *Gambar 2.2.3.38*	 *Gambar 2.2.3.39*
100	0	 *Gambar 2.2.3.40*	 *Gambar 2.2.3.41*
	1	 *Gambar 2.2.3.42*	 *Gambar 2.2.3.43*

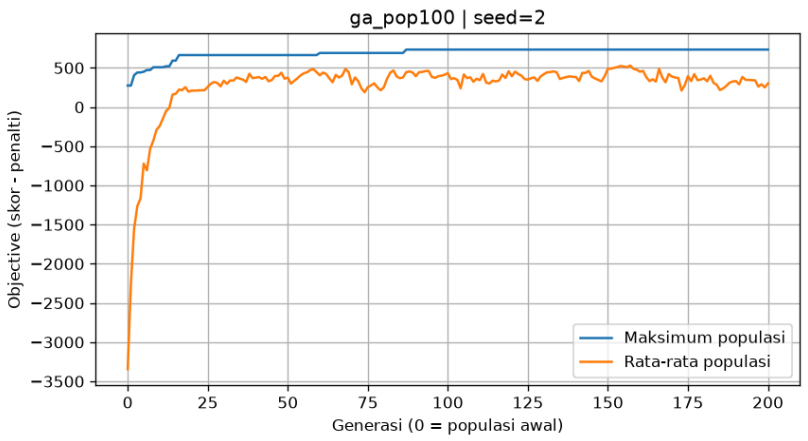
	2	 *Gambar 2.2.3.44*	 *Gambar 2.2.3.45*

Tabel 2.2.3.7 Kurva Fitness Maksimum dan Rata-Rata *Genetic Algorithm* dengan Variasi Ukuran Populasi

Populasi	Seed	Objective Curve
20	0	 *Gambar 2.2.3.46*
	1	 *Gambar 2.2.3.47*

	2	 *Gambar 2.2.3.48*
50	0	 *Gambar 2.2.3.49*
	1	 *Gambar 2.2.3.50*

	2	 *Gambar 2.2.3.51*
100	0	 *Gambar 2.2.3.52*
	1	 *Gambar 2.2.3.53*

	2	

Gambar 2.2.3.54

1. **Seberapa dekat dengan optimum global.**

   Hanya 1 dari 18 run GA yang mencapai 762 (pop 100, seed 1). Hasil terbaik berikutnya 747 (gap 2,0%) dicapai pada 2 run, dan rata-rata antar konfigurasi 696,3 sampai 733,3 (gap 3,8 sampai 8,6%). Dengan anggaran evaluasi yang kurang lebih setara (pop 50 x 201 generasi = 10.050 evaluasi, dibanding sekitar 8.700 evaluasi SA), GA lebih buruk daripada SA (720,7 vs 757,0). Dugaan kami, representasi dan crossover menjadi penyebab: placement tiap kendaraan di-encode terpisah padahal kelayakannya bergantung pada hubungan spasial antar kendaraan. One-point crossover yang menggabungkan separuh placement dari dua induk yang masing-masing rapi sering menghasilkan overlap, dan overlap itu dikenai penalti besar.

2. **Pengaruh banyak generasi.**

   Menaikkan generasi dari 50 ke 150 meningkatkan rata-rata dari 639,0 menjadi 720,7, dan hasil pada 150 dan 200 generasi identik di ketiga seed (733, 725, 704). Pada 400 generasi hasil naik lagi di semua seed (747, 733, 719; rata-rata 733,0), sehingga pencarian belum sepenuhnya stagnan pada generasi 200. Perbaikan dari 200 ke 400 generasi sebesar 12,3 poin rata-rata (gap 5,4% menjadi 3,8%) dengan waktu naik dari 0,77 menjadi 2,21 detik (sekitar 2,9 kali), sehingga hasil tambahan generasi makin mahal. Karena seed yang sama menghasilkan aliran bilangan acak yang sama, run 150, 200, dan 400 generasi kemungkinan mengikuti lintasan yang sama dan hanya berbeda pada panjangnya (hasil tiap seed tidak pernah turun karena elitisme), sehingga ketiga konfigurasi tidak sepenuhnya independen.

3. **Pengaruh jumlah populasi.**

Populasi yang lebih besar memperbaiki rata-rata: 696,3 (pop 20), 720,7 (pop 50), dan 733,3 (pop 100), dengan gap 8,6%, 5,4%, dan 3,8%. Hanya pop 100 yang pernah mencapai 762. Simpangan baku tidak turun monoton (45,2, 15,0, lalu 28,5), jadi populasi besar belum terbukti membuat hasil lebih stabil. Pop 20 memiliki hasil terendah (660) dan sebaran terlebar (660 sampai 747), yang konsisten dengan keragaman populasi yang sempit. Fenomena ini konsisten dengan keterbatasan keragaman genetik pada populasi kecil, yang menyebabkan algoritma rentan terjebak pada lokal optimum dan sangat sensitif terhadap inisialisasi awal. Waktu naik seiring ukuran populasi (0,17, 0,77, dan 2,70 detik). Secara teori biaya GA linear terhadap populasi x generasi, dan kenaikan yang sedikit melebihi linear pada data ini kemungkinan dipengaruhi beban mesin (durasi tiga seed pop 50 berkisar 0,55 sampai 1,14 detik).

4. **Konsistensi.**

   Hasil GA bervariasi antar seed (simpangan baku 14 sampai 45, rentang hasil hingga 87 poin pada pop 50 dengan 50 generasi) dan tidak ada konfigurasi yang konsisten mencapai optimum. Nilai akhir yang paling sering muncul adalah 733 (4 dari 18 run), diikuti 747, 725, 704, dan 682 (masing-masing 2 run).

2.2.4 Perbandingan Antar Algoritma

Rangkuman seluruh algoritma pada case1 (tiga seed per algoritma; untuk GA dipakai pop 50 dengan 200 generasi sebagai konfigurasi baku dan pop 100 sebagai konfigurasi terbaik).

Tabel 2.2.3.8 Perbandingan Kinerja Algoritma pada Case1 dengan Mode Fee

Algoritma	**Rata-rata final**	**Simpangan baku**	**Min – Maks**	**Gap ke 762**	**Rata-rata durasi (s)**
Random Restart HC	762,0	0,0	762 – 762	0%	6,63
Simulated Annealing	757,0	8,7	747 – 762	0,7%	7,52
Sideways Move HC	753,3	7,5	749 – 762	1,1%	11,66
GA (pop 100)	733,3	28,5	705 – 762	3,8%	2,70
GA (pop 50)	720,7	15,0	704 – 733	5,4%	0,77

Steepest Ascent HC	707,3	47,4	679 – 762	7,2%	0,78
Stochastic HC	683,0	43,6	653 – 733	10,4%	0,12

Random Restart memberi hasil terbaik dan paling konsisten (762 di ketiga seed) dengan waktu sekitar 6,6 detik. Simulated Annealing menyusul dengan 2 dari 3 seed mencapai optimum, dengan waktu pencarian hanya sekitar 0,35 detik, kira-kira 19 kali lebih cepat dari Random Restart (7,5 detik jika pengecekan local optimum ikut dihitung). Dari sisi jumlah evaluasi, SA memakai sekitar 8,7 ribu evaluasi, sedangkan Random Restart sekitar 200 ribu dan Sideways Move sekitar 300 ribu, sehingga SA adalah yang paling efisien per evaluasi. Sideways Move memberi hasil baik tetapi paling lambat di antara varian Hill-Climbing (11,7 detik). GA memberi hasil menengah dengan waktu 0,77 sampai 2,70 detik dan hanya sekali mencapai 762. Stochastic HC paling cepat (0,12 detik) tetapi hasilnya paling rendah, dan Steepest Ascent paling mudah terjebak local optimum (hanya 1 dari 3 seed mencapai 762). Seluruh kesimpulan ini berlaku untuk satu case dan hanya tiga seed per algoritma, sehingga perbedaan kecil (misalnya antara Random Restart dan Simulated Annealing, atau antara Steepest dan GA pop 50) belum dapat dianggap signifikan secara statistik

2.2.5 Pengujian Bonus

Subbab ini menguji dua spesifikasi bonus pada implementasi: multi-kapal dan objective function alternatif. Parameter algoritma sama dengan eksperimen sebelumnya (seed 0, 1, 2; GA memakai populasi 50 dan 200 generasi). Durasi SA yang ditampilkan adalah durasi pencarian murni (tanpa pengecekan local optimum). Gap dihitung terhadap nilai optimum global, yang dibuktikan dengan cara yang sama seperti pada case1: relaksasi yang hanya memperhatikan berat memberi batas atas, dan nilai tersebut tercapai oleh solusi yang valid.

1. Multikapal (cek data/case_multi.json)

   Kasus ini memuat 6 kendaraan (total tarif 254) dan 2 kapal berukuran 10 x 20 dengan MaxCapacity masing-masing 30. Enumerasi seluruh penugasan kendaraan ke kapal 0, kapal 1, atau di luar (3^6 = 729 kemungkinan) dengan batasan kapasitas saja menghasilkan batas atas 239, dan nilai ini dicapai oleh solusi valid, sehingga 239 adalah optimum global. Solusi optimum tidak mengangkut V0 (tarif 15, berat 7).

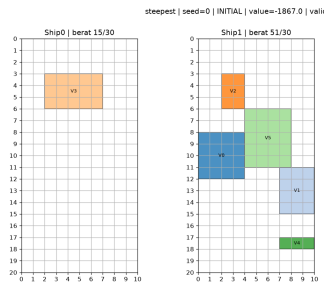
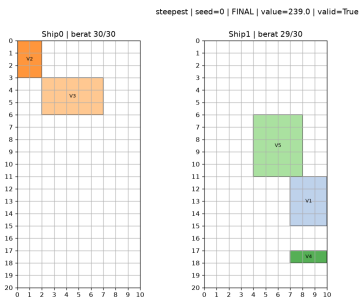
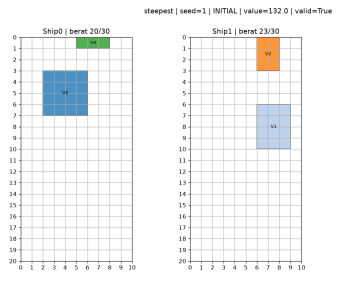
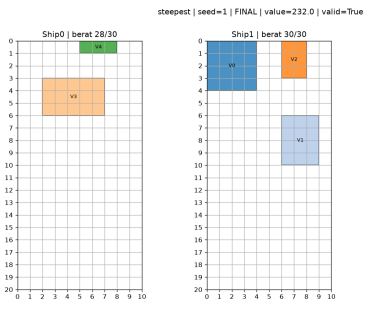
   Tabel 2.2.3.9 Hasil Pengujian Algoritma pada Kasus Multikapal

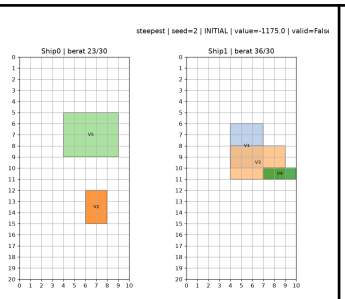
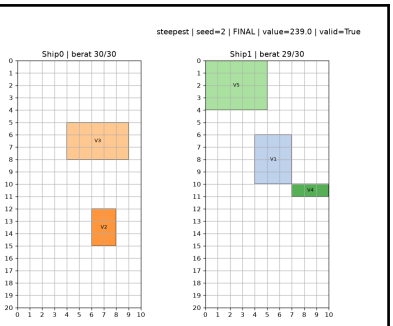
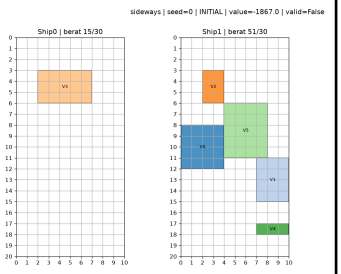
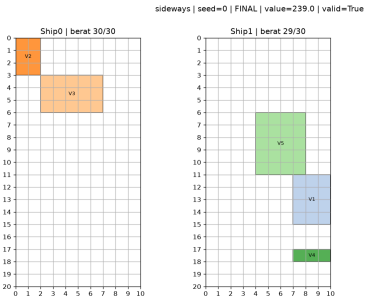
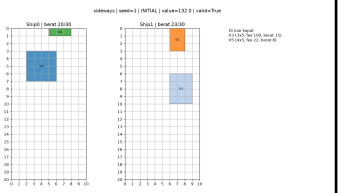
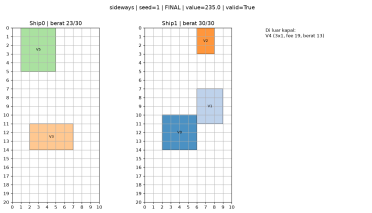
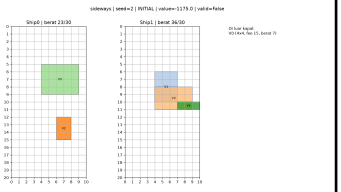
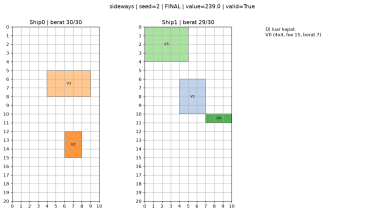
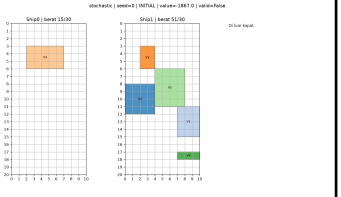
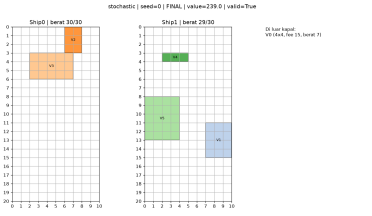
Algoritma	Seed 0	Seed 1	Seed 2	Rata-rata	Gap ke 239	Durasi rata-rata (s)
Steepest Ascent	239	232	239	236,7	1,0%	0,08

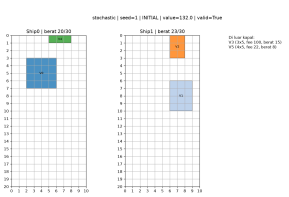
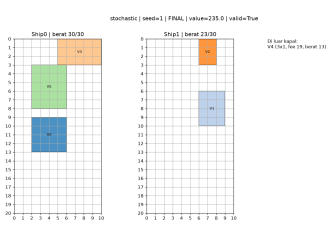
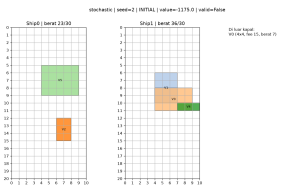
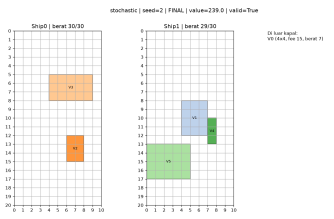
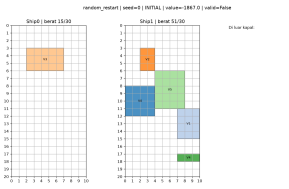
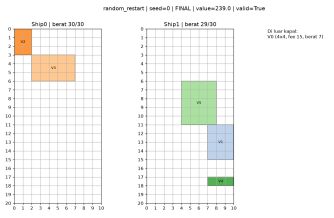
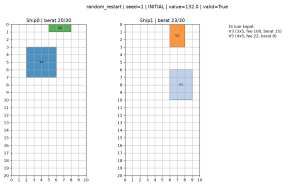
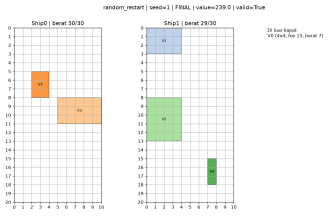
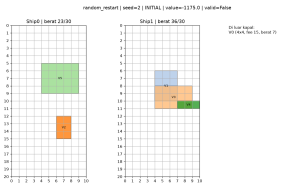
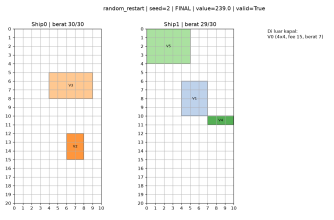
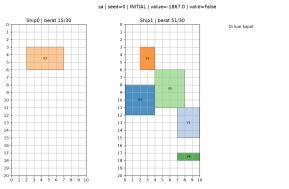
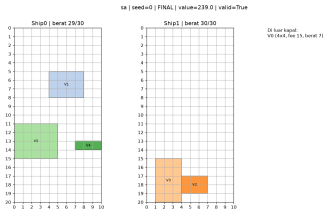
Sideways Move	239	235	239	237,7	0,6%	2,97
Stochastic	239	235	239	237,7	0,6%	0,05
Random Restart	239	239	239	239,0	0%	1,11
Simulated Annealing	239	239	239	239,0	0%	5,12
GA (pop 50, 200 gen)	239	239	235	237,7	0,6%	0,24

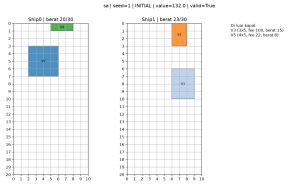
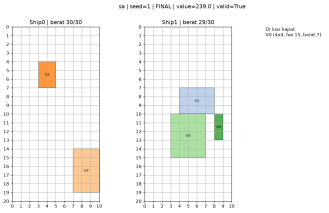
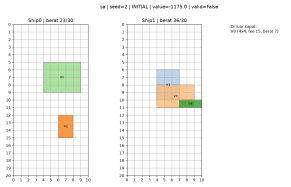
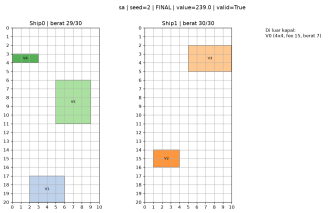
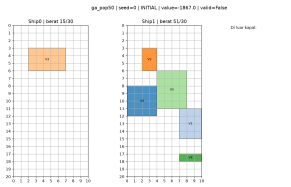
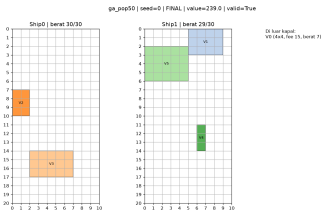
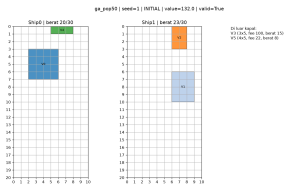
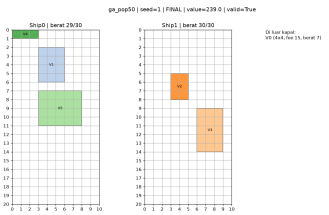
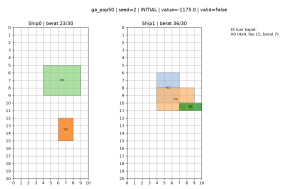
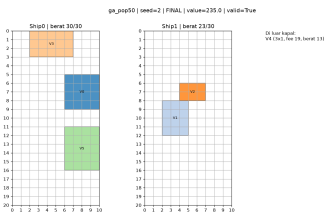
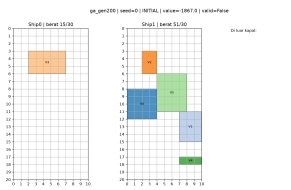
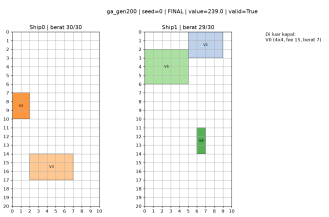
Seluruh hasil valid. Pada Random Restart seed 0, V2 dan V3 ditempatkan pada Ship0 (berat 30/30), V1, V4, dan V5 pada Ship1 (berat 29/30), dan V0 berada di luar kapal. Kapasitas diperiksa per kapal. Kasus ini kecil sehingga hampir semua algoritma mencapai optimum pada sebagian besar seed. Perbedaan antar algoritma lebih jelas pada case1. Meski demikian, urutan kualitasnya konsisten: Random Restart dan SA stabil di 239, sedangkan Steepest, Sideways, Stochastic, dan GA masing-masing gagal pada satu seed.

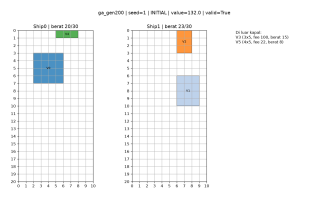
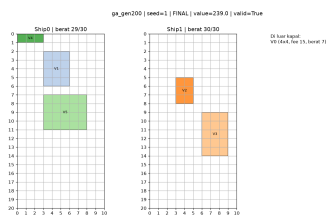
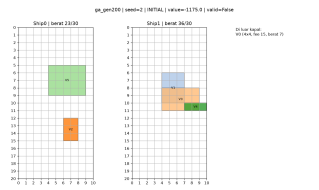
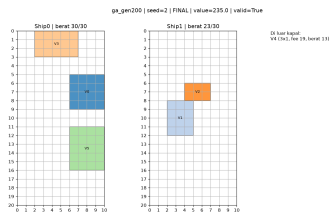
Tabel 2.2.3.10 Visualisasi State Awal dan Akhir pada Kasus Multikapal

Varian	Seed	Initial State	Final State
Steepest Ascent	0	 *Gambar 2.2.5.1*	 *Gambar 2.2.5.2*
	1	 *Gambar 2.2.5.3*	 *Gambar 2.2.5.4*

	2	 *Gambar 2.2.5.5*	 *Gambar 2.2.5.6*
Sideways Move (max 100)	0	 *Gambar 2.2.5.7*	 *Gambar 2.2.5.8*
	1	 *Gambar 2.2.5.9*	 *Gambar 2.2.5.10*
	2	 *Gambar 2.2.5.11*	 *Gambar 2.2.5.12*
Stochastic	0	 *Gambar 2.2.5.13*	 *Gambar 2.2.5.14*

	1	 *Gambar 2.2.5.15*	 *Gambar 2.2.5.16*
	2	 *Gambar 2.2.5.17*	 *Gambar 2.2.5.18*
Random Restart (10 restart)	0	 *Gambar 2.2.5.19*	 *Gambar 2.2.5.20*
	1	 *Gambar 2.2.5.21*	 *Gambar 2.2.5.22*
	2	 *Gambar 2.2.5.23*	 *Gambar 2.2.5.24*
Simulated Annealing	0	 *Gambar 2.2.5.25*	 *Gambar 2.2.5.26*

	1	 *Gambar 2.2.5.27*	 *Gambar 2.2.5.28*
	2	 *Gambar 2.2.5.29*	 *Gambar 2.2.5.30*
GA Pop 50	0	 *Gambar 2.2.5.31*	 *Gambar 2.2.5.32*
	1	 *Gambar 2.2.5.33*	 *Gambar 2.2.5.34*
	2	 *Gambar 2.2.5.35*	 *Gambar 2.2.5.36*
GA Gen 200	0	 *Gambar 2.2.5.37*	 *Gambar 2.2.5.38*

	1	 *Gambar 2.2.5.39*	 *Gambar 2.2.5.40*
	2	 *Gambar 2.2.5.41*	 *Gambar 2.2.5.42*

2. Objective Function

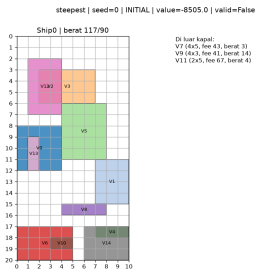
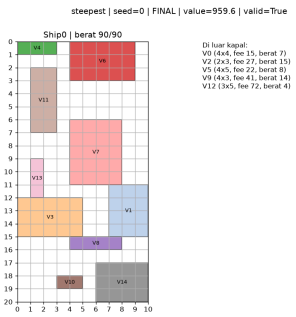
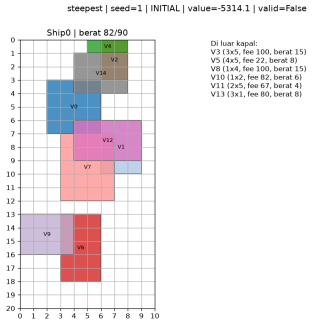
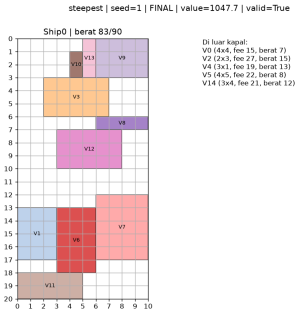
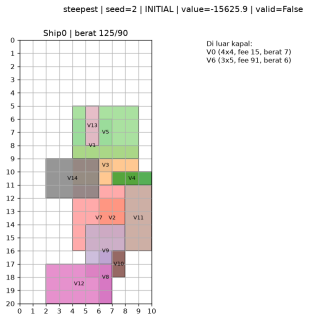
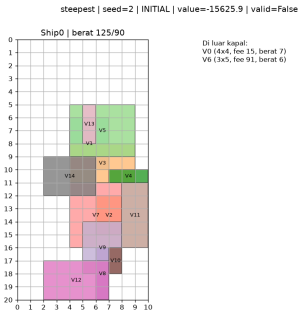
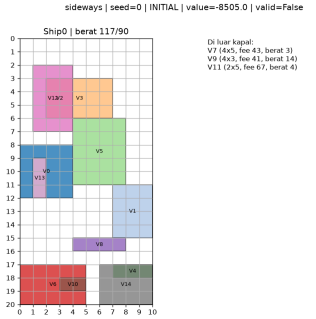
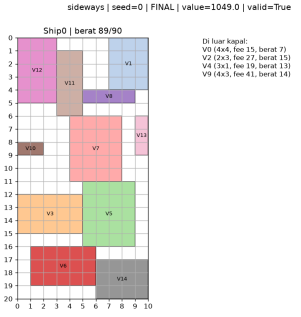
Mode `urgency` memakai skor fee x (1 + 1/(1 + ETA)), sehingga kendaraan dengan ETA lebih dekat bernilai lebih besar. Mode `area` memakai skor fee + 0,5 x w x l, sehingga kendaraan yang lebih luas bernilai lebih besar. Keduanya hanya memakai atribut yang sudah ada. Nilai optimum global ditentukan dengan knapsack 0/1 pada skor tiap mode dengan kapasitas 90 (hanya memperhatikan berat). Batas atas ini dicapai oleh solusi valid, yaitu 1.064,57 untuk `urgency` dan 824 untuk `area.`

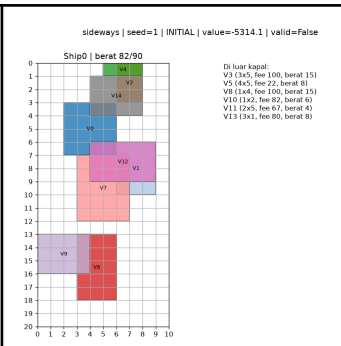
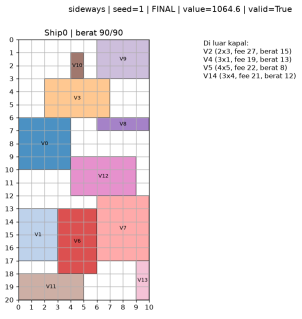
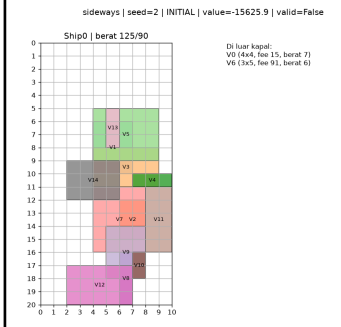
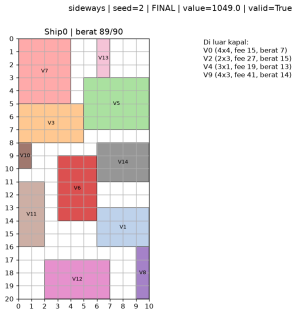
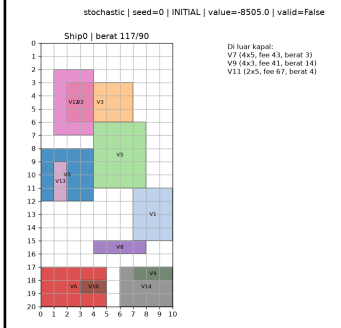
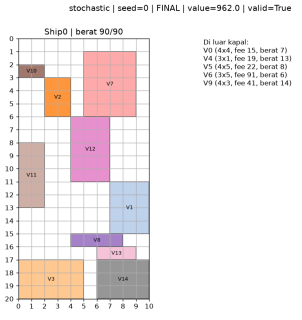
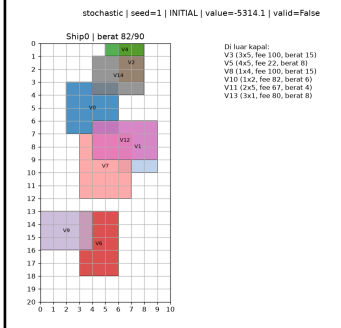
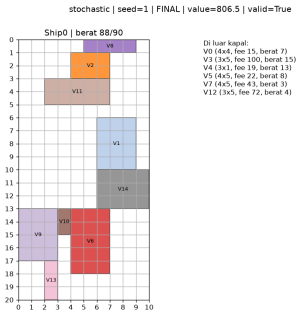
Mode `urgency` (optimum 1.064,57):

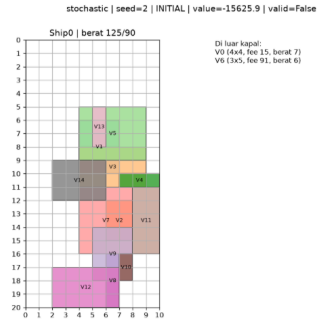
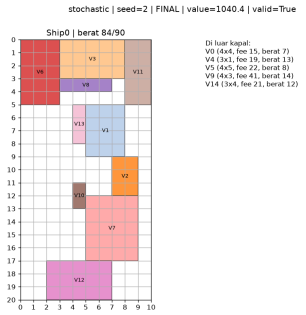
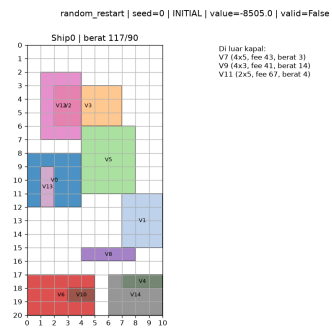
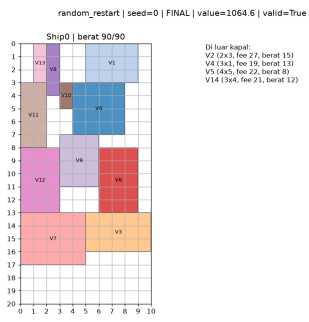
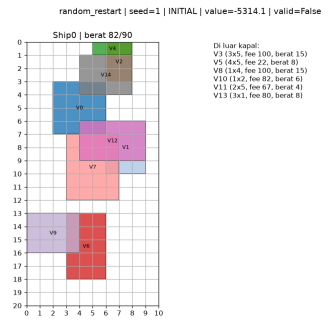
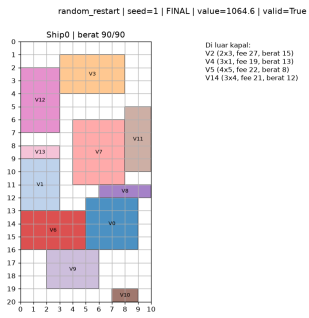
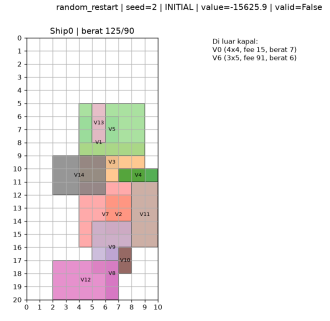
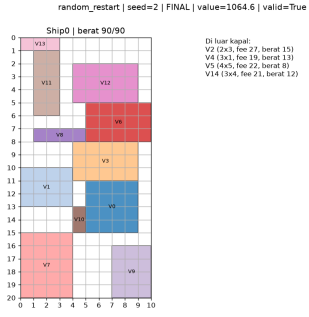
Tabel 2.2.3.11 Hasil Pengujian Algoritma dengan *Objective Function Mode Urgency*

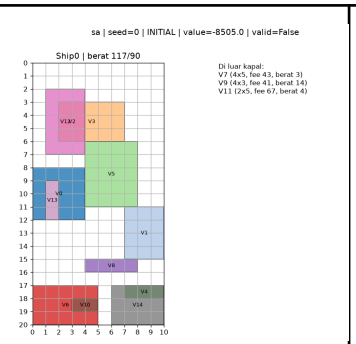
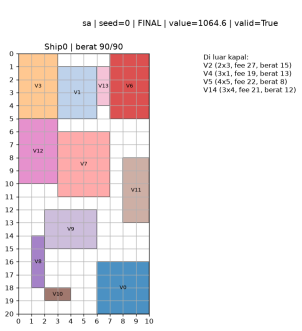
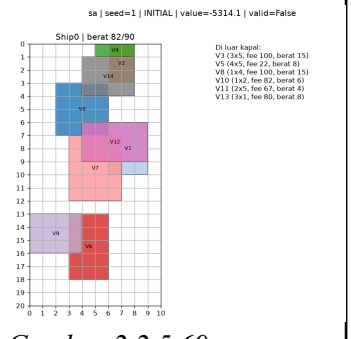
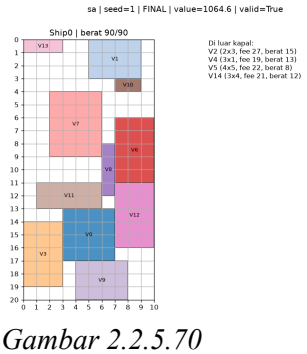
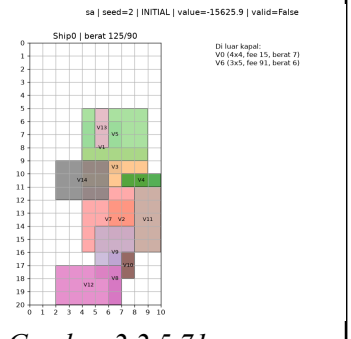
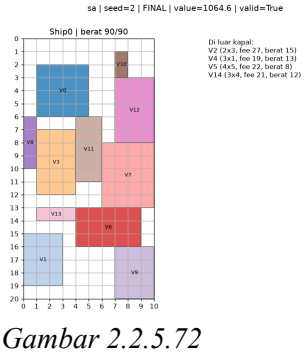
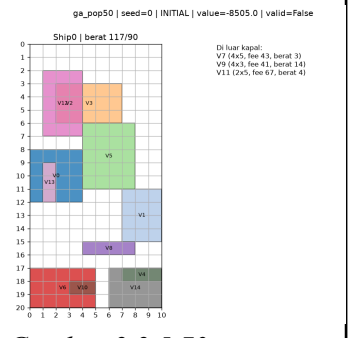
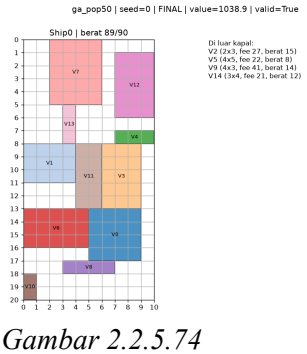
Algoritma	Seed 0	Seed 1	Seed 2	Rata-rata	Gap	Durasi rata-rata (s)
Steepest Ascent	959,62	1.047,69	973,77	993,69	6,7%	0,67
Sideways Move	1.049,00	1.064,57	1.049,00	1.054,19	1,0%	10,15
Stochastic	961,98	806,46	1.040,36	936,27	12,1%	0,07
Random Restart	1.064,57	1.064,57	1.064,57	1.064,57	0%	6,27
Simulated Annealing	1.064,57	1.064,57	1.064,57	1.064,57	0%	9,05
GA (pop 50, 200 gen)	1.038,90	936,02	1.040,36	1.005,09	5,6%	0,38

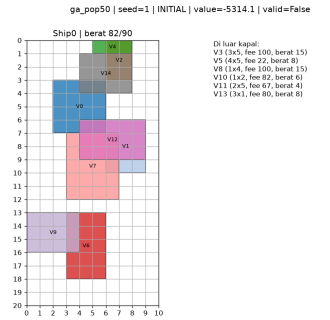
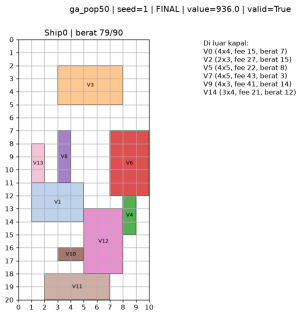
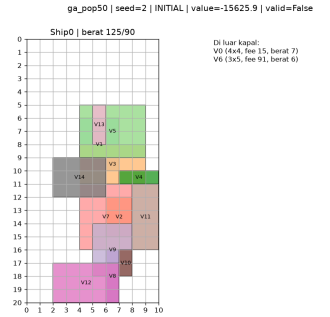
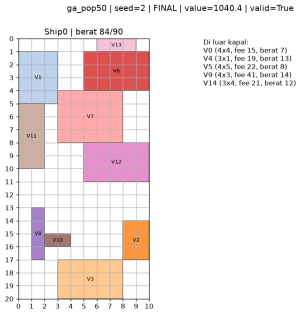
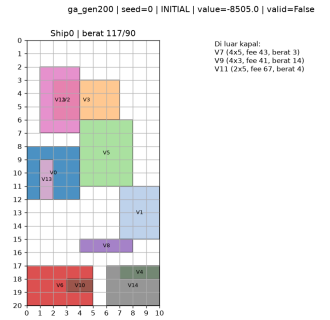
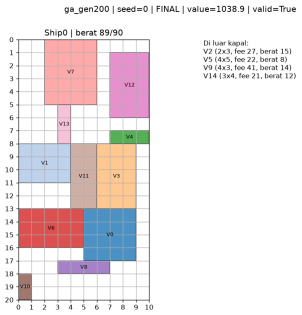
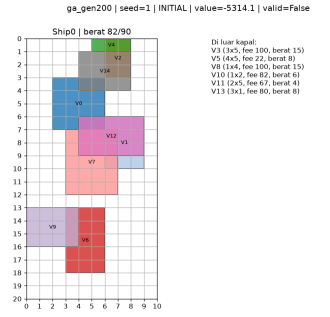
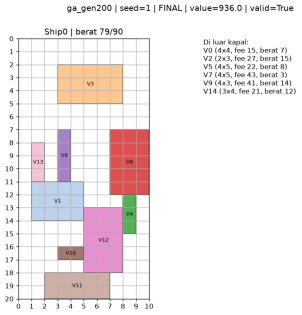
Tabel 2.2.3.12 Visualisasi State Awal dan Akhir pada Mode Urgency

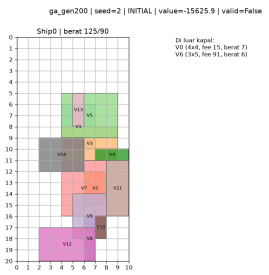
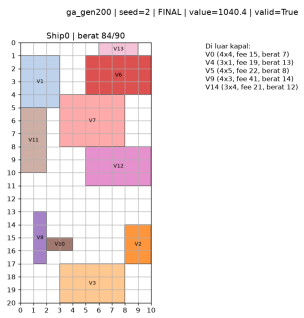
Varian	Seed	Initial State	Final State
Steepest Ascent	0	 *Gambar 2.2.5.43*	 *Gambar 2.2.5.44*
	1	 *Gambar 2.2.5.45*	 *Gambar 2.2.5.46*
	2	 *Gambar 2.2.5.47*	 *Gambar 2.2.5.48*
Sideways Move (max 100)	0	 *Gambar 2.2.5.49*	 *Gambar 2.2.5.50*

	1	 *Gambar 2.2.5.51*	 *Gambar 2.2.5.52*
	2	 *Gambar 2.2.5.53*	 *Gambar 2.2.5.54*
Stochastic	0	 *Gambar 2.2.5.55*	 *Gambar 2.2.5.56*
	1	 *Gambar 2.2.5.57*	 *Gambar 2.2.5.58*

	2	 *Gambar 2.2.5.59*	 *Gambar 2.2.5.60*
Random Restart (10 restart)	0	 *Gambar 2.2.5.61*	 *Gambar 2.2.5.62*
	1	 *Gambar 2.2.5.63*	 *Gambar 2.2.5.64*
	2	 *Gambar 2.2.5.65*	 *Gambar 2.2.5.66*

Simulated Annealing	0	 *Gambar 2.2.5.67*	 *Gambar 2.2.5.68*
	1	 *Gambar 2.2.5.69*	 *Gambar 2.2.5.70*
	2	 *Gambar 2.2.5.71*	 *Gambar 2.2.5.72*
GA Pop 50	0	 *Gambar 2.2.5.73*	 *Gambar 2.2.5.74*

	1	 *Gambar 2.2.5.75*	 *Gambar 2.2.5.76*
	2	 *Gambar 2.2.5.77*	 *Gambar 2.2.5.78*
GA Gen 200	0	 *Gambar 2.2.5.79*	 *Gambar 2.2.5.80*
	1	 *Gambar 2.2.5.81*	 *Gambar 2.2.5.82*

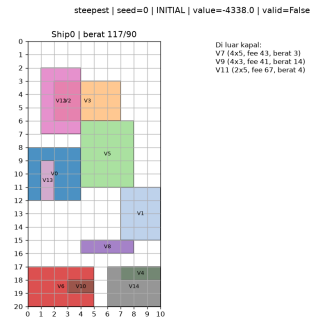
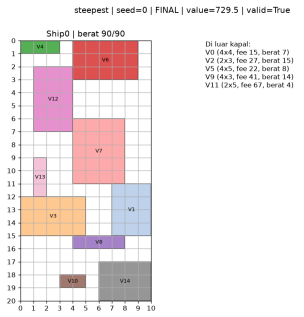
	2	 *Gambar 2.2.5.83*	 *Gambar 2.2.5.84*

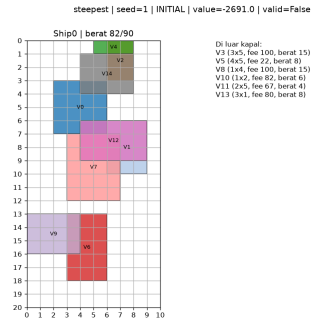
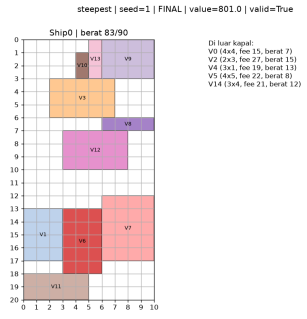
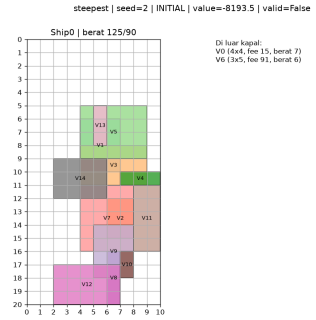
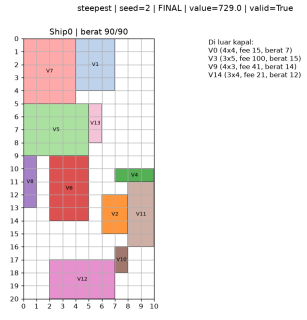
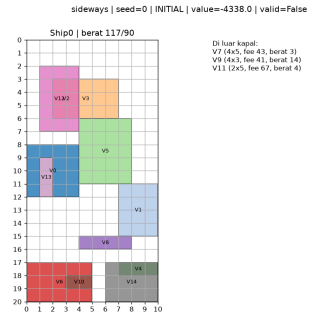
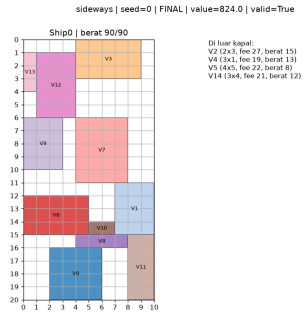
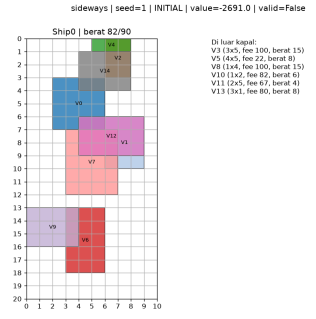
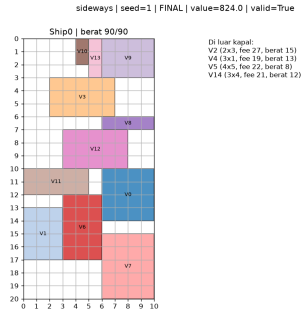
Mode `area` (optimum 824):

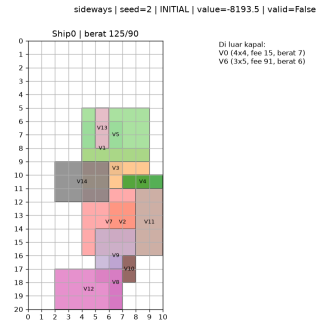
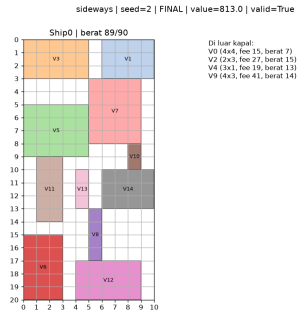
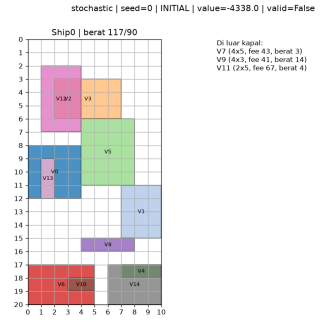
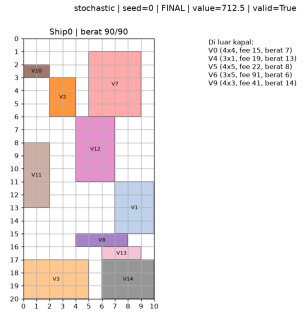
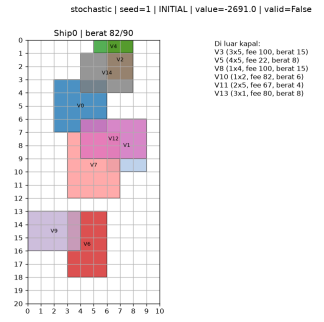
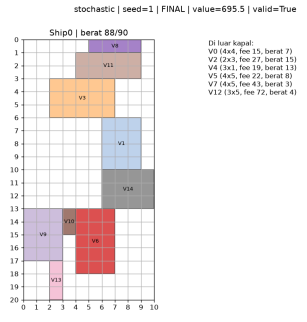
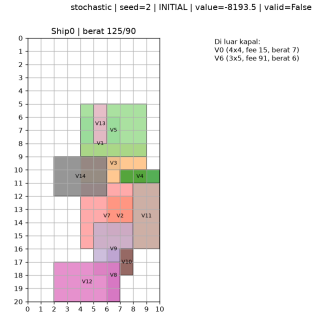
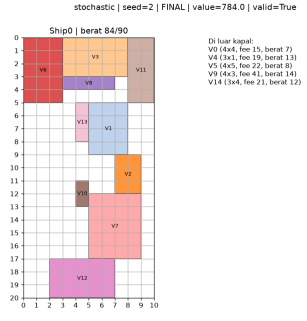
Tabel 2.2.3.13 Hasil Pengujian Algoritma dengan *Objective Function Mode Area*

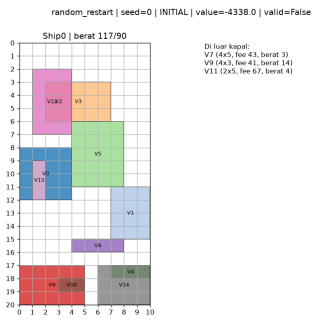
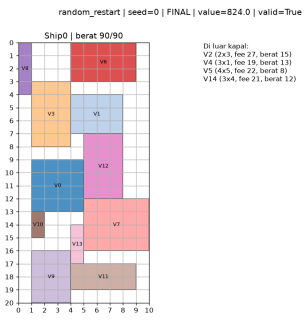
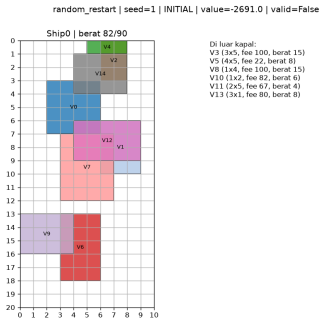
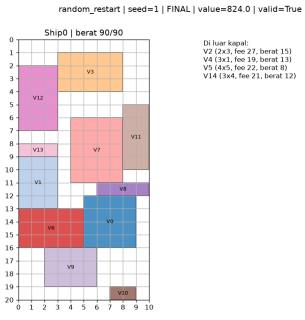
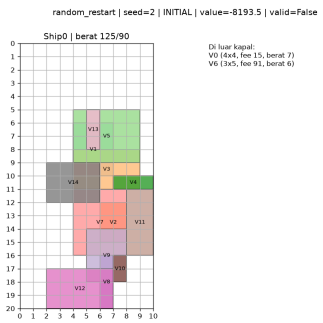
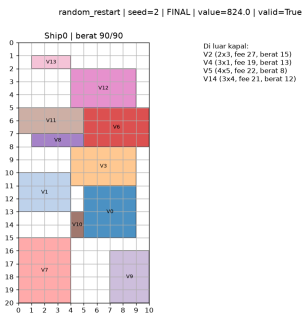
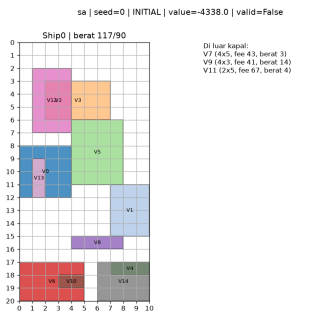
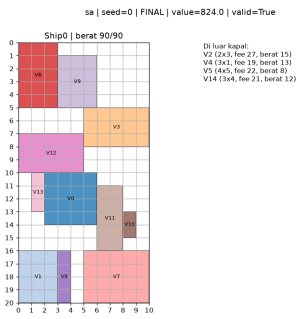
Algoritma	Seed 0	Seed 1	Seed 2	Rata-rata	Gap	Durasi rata-rata (s)
Steepest Ascent	729,50	801,00	729,00	753,17	8,6%	0,63
Sideways Move	824,00	824,00	813,00	820,33	0,4%	10,31
Stochastic	712,50	695,50	784,00	730,67	11,3 %	0,09
Random Restart	824,00	824,00	824,00	824,00	0%	6,31
Simulated Annealing	824,00	824,00	824,00	824,00	0%	10,22
GA (pop 50, 200 gen)	784,00	753,50	781,00	772,83	6,2%	0,38

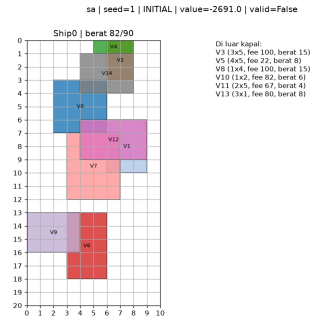
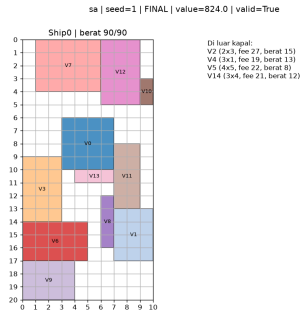
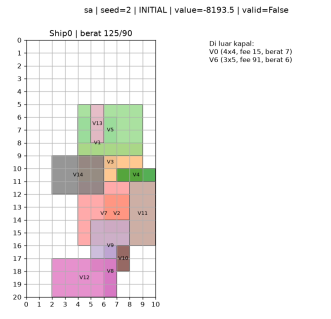
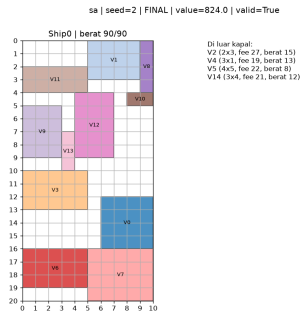
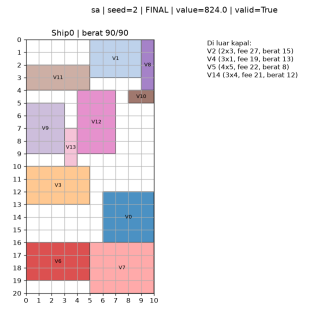
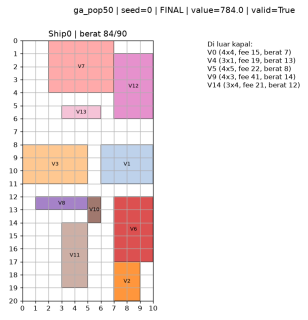
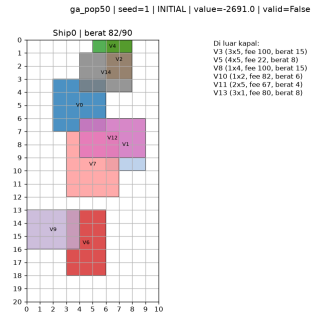
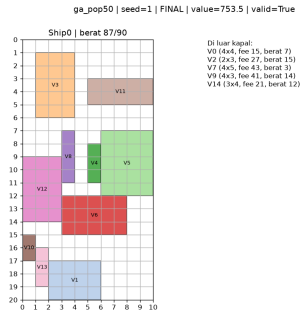
Tabel 2.2.3.14 Visualisasi State Awal dan Akhir pada Mode Area

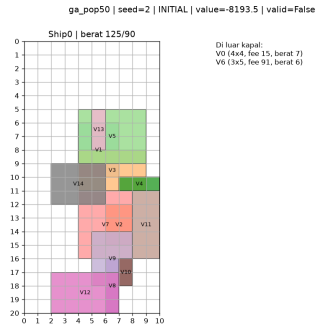
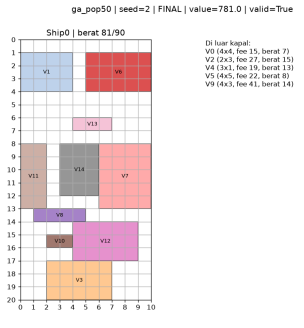
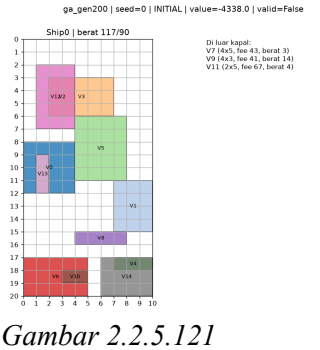
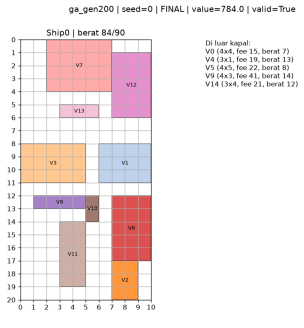
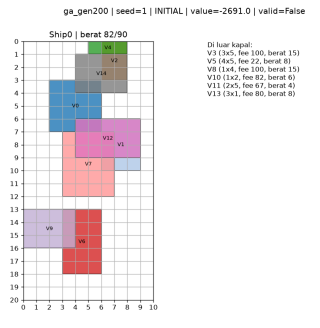
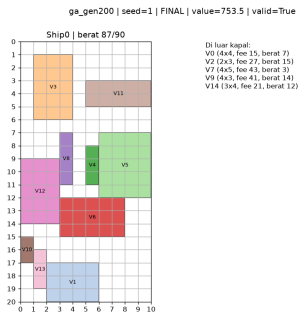
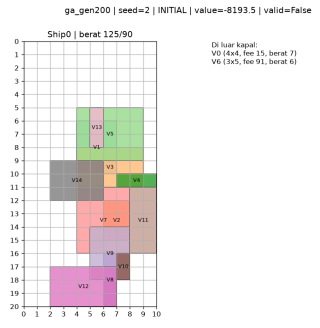
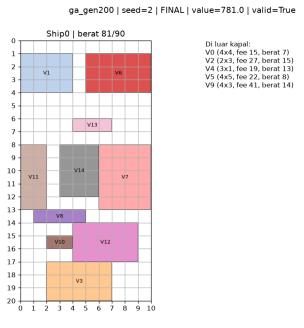
Varian	Seed	Initial State	Final State
Steepest Ascent	0	 *Gambar 2.2.5.85*	 *Gambar 2.2.5.86*

	1	 *Gambar 2.2.5.87*	 *Gambar 2.2.5.88*
	2	 *Gambar 2.2.5.89*	 *Gambar 2.2.5.90*
Sideways Move (max 100)	0	 *Gambar 2.2.5.91*	 *Gambar 2.2.5.92*
	1	 *Gambar 2.2.5.93*	 *Gambar 2.2.5.94*

	2	 *Gambar 2.2.5.95*	 *Gambar 2.2.5.96*
Stochastic	0	 *Gambar 2.2.5.97*	 *Gambar 2.2.5.98*
	1	 *Gambar 2.2.5.99*	 *Gambar 2.2.5.100*
	2	 *Gambar 2.2.5.101*	 *Gambar 2.2.5.102*

Random Restart (10 restart)	0	 *Gambar 2.2.5.103*	 *Gambar 2.2.5.104*
	1	 *Gambar 2.2.5.105*	 *Gambar 2.2.5.106*
	2	 *Gambar 2.2.5.107*	 *Gambar 2.2.5.108*
Simulated Annealing	0	 *Gambar 2.2.5.109*	 *Gambar 2.2.5.110*

	1	 *Gambar 2.2.5.111*	 *Gambar 2.2.5.112*
	2	 *Gambar 2.2.5.113*	 *Gambar 2.2.5.114*
GA Pop 50	0	 *Gambar 2.2.5.115*	 *Gambar 2.2.5.116*
	1	 *Gambar 2.2.5.117*	 *Gambar 2.2.5.118*

	2	 *Gambar 2.2.5.119*	 *Gambar 2.2.5.120*
GA Gen 200	0	 *Gambar 2.2.5.121*	 *Gambar 2.2.5.122*
	1	 *Gambar 2.2.5.123*	 *Gambar 2.2.5.124*
	2	 *Gambar 2.2.5.125*	 *Gambar 2.2.5.126*

Seluruh hasil valid. Urutan kualitas algoritma pada kedua mode sama dengan mode fee: Random Restart dan SA konsisten mencapai optimum, Sideways Move mendekati optimum, sedangkan Steepest, GA, dan Stochastic tertinggal. Ini menunjukkan bahwa perilaku algoritma ditentukan oleh struktur landscape (penalti dan local optimum), bukan oleh definisi skor. Penalti per satuan pelanggaran menyesuaikan diri dengan skor tertinggi tiap mode, sehingga constraint tetap lebih

mahal daripada keuntungan kendaraan mana pun. Nilai pada mode berbeda tidak dapat dibandingkan langsung karena skalanya berbeda.

BAB III: KESIMPULAN DAN SARAN

3.1 Kesimpulan

Permasalahan pemilihan dan penempatan kendaraan pada geladak kapal telah diimplementasikan menggunakan empat varian Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Representasi state memuat kapal, posisi, dan orientasi setiap kendaraan, sedangkan fungsi objektif memaksimalkan total tarif pengiriman dengan penalti terhadap pelanggaran batasan. Penyimpanan solusi feasible terbaik memungkinkan program mengembalikan konfigurasi yang memenuhi batasan meskipun pencarian melewati state yang tidak valid. Pada pengujian case1 dalam mode fee, seluruh algoritma menghasilkan solusi akhir tanpa tumpang tindih, berada di dalam batas geladak, dan tidak melebihi kapasitas angkut kapal. Implementasi juga diuji pada kasus dua kapal serta pada objective alternatif `urgency` dan `area;` pola perbandingan antar algoritma tetap sama dan seluruh solusi akhir valid (subbab 2.2.5).

Random Restart Hill-Climbing menghasilkan kualitas solusi terbaik dan paling konsisten pada pengujian ini. Ketiga seed mencapai nilai optimum global sebesar 762. Simulated Annealing mencapai optimum pada dua dari tiga seed dengan rata-rata nilai akhir 757,0, sedangkan Sideways Move Hill-Climbing memperoleh rata-rata 753,3. Steepest Ascent dan Stochastic Hill-Climbing masing-masing memperoleh rata-rata 707,3 dan 683,0. Hasil tersebut menunjukkan bahwa memulai pencarian dari beberapa state awal maupun mengizinkan langkah yang tidak langsung meningkatkan nilai objektif dapat membantu pencarian pada kasus yang diuji, tetapi tidak menjamin optimum tercapai pada setiap run.

Pada Genetic Algorithm, kualitas solusi dipengaruhi oleh jumlah generasi dan ukuran populasi. Dengan populasi tetap sebesar 50, peningkatan jumlah generasi dari 50 menjadi 150 menaikkan rata-rata nilai akhir dari 639,0 menjadi 720,7. Hasil pada 200 generasi masih sama, sedangkan 400 generasi meningkatkan rata-rata menjadi 733,0. Pada pengujian dengan 200 generasi, peningkatan populasi dari 20 menjadi 50 dan 100 menghasilkan rata-rata berturut-turut sebesar 696,3, 720,7, dan 733,3. Meskipun demikian, hanya satu dari 18 run GA yang mencapai nilai 762, yaitu konfigurasi populasi 100 dan 200 generasi pada seed 1. Penambahan populasi dan generasi memperbaiki rata-rata hasil dalam pengujian ini, tetapi belum menjamin tercapainya optimum maupun peningkatan kestabilan hasil antar-seed.

Kualitas solusi juga perlu dipertimbangkan bersama waktu komputasi. Dalam tabel perbandingan, Stochastic Hill-Climbing memiliki rata-rata durasi paling singkat, yaitu 0,12 detik, tetapi menghasilkan rata-rata nilai akhir terendah. Random Restart Hill-Climbing membutuhkan rata-rata 6,63 detik untuk mencapai optimum pada seluruh seed. Sideways

Move Hill-Climbing membutuhkan waktu paling lama di antara varian Hill-Climbing, yaitu 11,66 detik, meskipun rata-rata hasilnya lebih rendah daripada Random Restart. Durasi Waktu pencarian murni Simulated Annealing hanya sekitar 0,35 detik (7,5 detik jika pemeriksaan local optimum ikut dihitung).

Berdasarkan hasil tersebut, Random Restart Hill-Climbing sesuai apabila prioritas adalah nilai akhir tertinggi yang konsisten pada seluruh seed yang diuji (6,63 detik). Simulated Annealing sesuai apabila waktu komputasi menjadi pertimbangan, karena mencapai rata-rata 757,0 (gap 0,7%) dengan waktu sekitar 0,35 detik. Pemilihan algoritma untuk kasus lain tetap perlu mempertimbangkan kualitas solusi, waktu komputasi, dan parameter yang digunakan. Seluruh temuan dalam laporan ini terbatas pada satu kasus dan tiga seed per konfigurasi, sehingga belum dapat digeneralisasi sebagai keunggulan mutlak suatu algoritma untuk seluruh permasalahan penempatan kendaraan.

3.2 Saran

Pengujian selanjutnya disarankan menggunakan lebih banyak kasus dengan variasi jumlah dan ukuran kendaraan, kapasitas serta dimensi geladak, dan jumlah kapal. Jumlah seed juga perlu ditambah agar kestabilan hasil dapat dinilai dengan lebih baik. Pengujian tersebut dapat menunjukkan apakah pola perbandingan algoritma pada case1 tetap berlaku ketika ukuran dan karakteristik permasalahan berubah.

Eksperimen parameter dapat diperluas secara terkontrol, terutama terhadap probabilitas crossover, probabilitas mutasi, dan jumlah elit pada Genetic Algorithm serta suhu awal dan laju pendinginan pada Simulated Annealing. Setiap eksperimen sebaiknya mengubah satu parameter sementara parameter lain dipertahankan. Untuk pengembangan GA, mekanisme perbaikan solusi setelah crossover atau mutasi dapat diuji sebagai tambahan agar pelanggaran batasan berkurang. Dampaknya perlu dievaluasi terhadap kualitas solusi dan waktu komputasi sebelum diterapkan sebagai bagian dari algoritma.

Perbandingan efisiensi perlu membedakan waktu pencarian, waktu pemeriksaan local optimum, dan waktu total algoritma secara konsisten. Jumlah evaluasi fungsi objektif juga sebaiknya mencakup evaluasi tambahan untuk pemeriksaan tersebut. Dengan pencatatan ini, peningkatan kualitas solusi dapat dibandingkan dengan biaya komputasi yang benar-benar digunakan oleh masing-masing algoritma.

PEMBAGIAN TUGAS

Nama	NIM	Tugas
Danya Damaris	18224024	1. Laporan Bab I 2. Algoritma Hill-Climbing 3. Algoritma Simulated Annealing 4. Menyempurnakan Web Dashboard
Lukman Hakim Syah Ardhana	18224097	1. Laporan Bab III 2. Merapikan Laporan Bab II 3. Genetic Algorithm - Roulette Wheel 4. Melakukan Integrasi Akhir
Jacko Luciano	18224101	1. Laporan Bab II 2. Kode di folder Core (Fondasi) 3. Membuat web visualization untuk simulasi 4. Implementasi multikapal 5. Implementasi Objective function alternatif (bonus 1)

REFERENSI

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Lodi, A., Martello, S., & Monaci, M. (2002). Two-dimensional packing problems: A survey. European Journal of Operational Research, 141(2), 241–252.

Goldberg, D. E. (1989). Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. Science, 220(4598), 671–680.

LAMPIRAN

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	: Jacko Luciano/18224101, Lukman Hakim Syah Ardhana/18224097, Danya Damaris/18224024
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah	: IF3070
Nama Mata Kuliah	: Dasar Inteligensi Artifisial
Judul Tugas/Proposal	: Tugas Besar 1/Optimasi Penempatan Kendaraan pada Geladak Kapal Kargo Menggunakan Algoritma Local Search: Hill-Climbing, Simulated Annealing, dan Genetic Algorithm

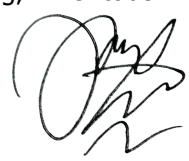
Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Level 2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Level 2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan: Implementasi simulasi web menggunakan tools Claude AI	Ya	Level 3

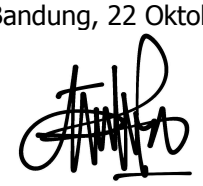
Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



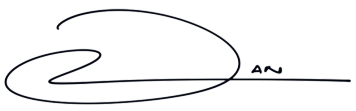
Jacko Luciano
18224101

Bandung, 22 Oktober 2026



Lukman Hakim S. A
18224097

Bandung, 22 Oktober 2026



Danya Damaris
18224024